

FINAL YEAR PROJECT REPORT

Project title:

**Design and Development of a Real-Time Queue Monitoring and
Wait-Time Estimation System**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

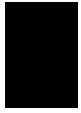
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	viii
List of Tables	x
List of Abbreviations and Symbols	xi
General Introduction	1
1 Context of the Project	2
1.1 Introduction	3
1.2 Host Organization	3
1.2.1 DataDoIt	3
1.2.2 Sectors of Activity	4
1.3 Presentation of the Project	5
1.3.1 Project Context	5
1.3.2 Problem Statement	5
1.3.3 Study of Existing Approaches	6
1.3.3.1 Queue Management Platforms	6
1.3.3.2 Sensor-Based Analytics	7
1.3.3.3 Vision-Based Monitoring Frameworks	7
1.3.3.4 Positioning of the Proposed Solution	7
1.3.4 Proposed Solution and Project Objectives	8
1.3.5 Project Scope, Boundaries, and Stakeholders	9
1.4 Project Management Methodology	10
1.4.1 Choice of Methodology: Agile	10

1.4.2	SCRUM	10
1.4.3	Sprint Planning and Execution	11
1.5	Conclusion	12
2	Requirements Specification and Analysis	13
2.1	Introduction	14
2.2	Requirements Specification	14
2.2.1	Identification of Actors	14
2.2.2	Functional Requirements	15
2.2.3	Non-functional Requirements	17
2.3	Product Backlog and Sprint Organization	17
2.4	Global Use Case Diagram	18
2.5	Project Environment	19
2.5.1	Hardware and Software Environment	19
2.5.2	Development Tools	19
2.5.3	Technologies Used	20
2.6	Software Architecture	21
2.6.1	Overall Architecture	21
2.6.2	Backend Architecture	23
2.6.3	Frontend Architecture	23
2.7	Conclusion	24
3	Sprint 1: Perception and Queue Analytics	25
3.1	Introduction	26
3.2	Sprint Objectives and Backlog	26
3.3	Sprint Backlog	26
3.4	Design	28
3.4.1	Perception and Analytics Flow	28

3.5	Implementation	30
3.5.1	Perception Pipeline	30
3.5.2	Queue Event Windows and Rate Estimation	32
3.5.3	Wait-Time Logic and Adaptive Estimation	32
3.5.4	Runtime Metric Contract	33
3.6	Conclusion	33
4	Sprint 2: Backend Services and API Integration	34
4.1	Introduction	35
4.2	Sprint Objectives and Backlog	35
4.3	Design	36
4.3.1	Backend Domain Model and Class Structure	36
4.4	Implementation	37
4.4.1	Source Integration and Camera Discovery	37
4.4.2	API and Live Transport	38
4.4.3	Persistence, State Recovery, and System Resilience	41
4.5	Conclusion	41
5	Sprint 3: Web Dashboard and Live Monitoring	43
5.1	Introduction	45
5.2	Sprint Objectives and Backlog	45
5.2.1	User Stories	45
5.2.2	Technical Tasks	47
5.3	Design	47
5.3.1	Visual Design and Interface Architecture	47
5.3.2	Dashboard Data Flow and Integration Architecture	48
5.3.3	Design Rationale	48
5.4	Implementation	50

5.4.1	Project Setup and Layout Skeleton	50
5.4.2	Styling and Reusable Interface Rules	50
5.4.3	API Integration	50
5.4.4	Authentication and Role-Based Navigation	50
5.4.5	Dashboard State Orchestration and Live Update Reliability	51
5.4.6	Live Video Delivery and Transport Selection	53
5.4.7	Dashboard UI Integration	54
5.4.8	Manager Onboarding Workflow	54
5.4.9	Analytics and Historical Reporting	56
5.4.10	Integration Testing	58
5.5	Conclusion	60
6	Sprint 4: Automation, Evaluation, and Optimization	61
6.1	Introduction	62
6.2	Sprint Objectives and Backlog	62
6.2.1	Technical Tasks	63
6.3	Design	64
6.3.1	Alert Automation and n8n Workflow	64
6.3.2	System Verification and Evaluation Strategy	65
6.4	Implementation	67
6.4.1	Alert Payload and Delivery Architecture	67
6.4.2	Telegram Integration and Notification Delivery	67
6.4.3	Asynchronous Alert Dispatch and Deduplication	69
6.4.4	n8n Workflow Structure and Validation	69
6.4.5	Comprehensive Integration Verification	69
6.4.6	Runtime Optimization and Performance Tuning	70
6.4.7	Fault-Tolerance and Recovery Mechanisms	70

6.4.8	System Integration and Operational Readiness	70
6.5	Conclusion	71
7	General Conclusion	72
7.1	Summary	73
7.2	Objective Review	73
7.3	Future Work	74



List of Figures

1.1	DataDoIt company logo	4
1.2	Scrum process used to organize the project.	11
2.1	Global use case diagram of the queue wait-time estimation system. . . .	18
2.2	Overall system architecture from video acquisition to alert delivery. . . .	22
3.1	Sprint 1 activity flow from frame ingestion to queue metrics output. . . .	29
3.2	Sprint 1 analytics activity diagram for queue metric generation.	31
4.1	Sprint 2 backend class diagram showing core entities for feed orchestra- tion and persistence.	37
4.2	Camera discovery activity diagram: ONVIF discovery, RTSP resolution, and stream validation.	39
4.3	ONVIF protocol sequence: WS-Discovery broadcast, direct media service resolution, and device service indirection.	40
5.1	Dashboard data flow sequence showing protected access (alt), metric broadcasts (loop), fallback polling (opt), and video transport negotiation (alt).	49
5.2	Manager sign-in screen for dashboard access.	51
5.3	Manager account creation screen.	52
5.4	Administrator manager-administration interface.	52
5.5	Manager dashboard: live surface, feed summaries, and recent activity. .	54

5.6	Manager onboarding activity diagram showing user actions and UI state transitions from setup wizard initialization through feed launch confirmation. Decision nodes show ONVIF versus upload path selection and the test validation retry loop.	55
5.7	Feed setup screen used to prepare an uploaded video or ONVIF camera source before zone tracing and model selection.	56
5.8	Zone tracing screen used to define the queue polygon for the selected feed.	57
5.9	Model selection screen used to choose the YOLO size before launching the feed.	58
5.10	Review and launch panel used to confirm staged feeds before batch creation or launch.	59
5.11	Analytics dashboard: KPI summaries, trends, zone rankings, and export.	60
6.1	Sprint 4 n8n workflow for routing queue alerts through cooldown checks, archival, and Telegram delivery.	65
6.2	Sprint 4 evaluation and resilience activity diagram for tests, tuning, and recovery verification.	66
6.3	Example Telegram alert delivered.	68



List of Tables

1.1	From operational pain points to system-level requirements	6
1.2	Positioning of the proposed solution against existing approaches	8
1.3	Objective-to-module traceability	9
1.4	Scrum module mapping used in this report	11
1.5	Sprint planning used in the report	12
2.1	Functional requirement traceability matrix	16
2.2	Non-functional requirement quality matrix	17
2.3	Primary global use cases	19
2.4	Hardware environment	19
2.5	Development tools used in the project	20
2.6	Technologies used in the project	21
3.1	Sprint 1 user stories	27
3.2	Sprint 1 technical tasks	28
4.1	Sprint 2 user stories	36
5.1	Sprint 3 user stories	46
5.2	Sprint 3 technical tasks	47
6.1	Sprint 4 user stories	63
6.2	Sprint 4 technical tasks	64



List of Abbreviations and Symbols

The following symbols and abbreviations are used throughout the report.

Symbols

λ	Arrival rate.
μ	Service rate.
W	Expected wait time.
L	Number of people currently in the queue zone.

Abbreviations

API

Application Programming Interface.

BFF

Backend for Frontend.

CSV

Comma-Separated Values.

HTTP

Hypertext Transfer Protocol.

MJPEG

Motion JPEG.

ONVIF

Open Network Video Interface Forum.

REST

Representational State Transfer.

RTSP

Real-Time Streaming Protocol.

TCP

Transmission Control Protocol.

UDP

User Datagram Protocol.

UI

User Interface.

WebRTC

Web Real-Time Communication.

WHEP

WebRTC HTTP Egress Protocol.

WS-Discovery

Web Services Discovery.



General Introduction

Service organizations are under constant pressure to reduce waiting time and improve service quality. Supermarkets, banks, and service centers all face the same problem: queues change quickly, are difficult to monitor by hand, and react strongly to demand fluctuations.

In practice, customers often join a queue without knowing how long they will wait, while managers and operational staff rarely have reliable real-time indicators for adjusting resources quickly. Manual observation is subjective, and simple counting tools are not enough to estimate waiting time in a meaningful way.

This end-of-studies project addresses that gap through the design and development of a real-time queue monitoring system based on computer vision and queueing theory. The proposed system detects and tracks people in video streams, analyzes occupancy inside configurable queue zones, estimates arrival and service rates, computes waiting time, and exposes metrics that can be used operationally.

It combines classical queueing theory with computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates.

This report is organized as follows. Chapter 1 presents the project context, host organization, problem statement, and objectives. Chapter 2 defines the requirements analysis and the main architecture choices. Chapters 3 to 6 follow the implementation sprint by sprint, from perception and queue analytics to frontend integration and operational automation. The general conclusion closes the report and presents future work directions.

Context of the Project

Table of Content

1.1	Introduction	3
1.2	Host Organization	3
1.2.1	DataDoIt	3
1.2.2	Sectors of Activity	4
1.3	Presentation of the Project	5
1.3.1	Project Context	5
1.3.2	Problem Statement	5
1.3.3	Study of Existing Approaches	6
1.3.4	Proposed Solution and Project Objectives	8
1.3.5	Project Scope, Boundaries, and Stakeholders	9
1.4	Project Management Methodology	10
1.4.1	Choice of Methodology: Agile	10
1.4.2	SCRUM	10
1.4.3	Sprint Planning and Execution	11
1.5	Conclusion	12

1.1 Introduction

This chapter situates the Queue Wait-Time Estimation System in its operational and organizational context. It explains why queue monitoring remains important for service organizations and why a complete technical solution is needed instead of isolated counting tools.

It also sets the baseline used throughout the report: the collaboration with DataDoIt, the problem statement, the comparison with existing approaches, the solution scope, and the Scrum organization of the work.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt works on projects that connect data science and sustainability for governments, municipalities, and enterprises across the EMEA region. Its services cover decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. The company uses the motto Measure It, Optimize It, Do It to describe a working method based on analysis, planning, and deployment. At the time of the project, it had delivered more than 15 active projects across 6 sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration provided a concrete setting for work centered on data collection, real-time monitoring, and automated analytics, and these concerns directly shaped the design and scope of the project, with the DataDoIt company logo shown in Figure 1.1.

The collaboration perimeter that framed the project centered on queue management in service scenarios, multi-source video ingestion, low-latency monitoring, actionable



Figure 1.1: DataDoIt company logo

queue indicators, and iterative validation across backend, API, frontend, and automation layers.

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,
- financial services firms navigating ESG compliance and green finance opportunities,
- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is often subjective, hard to scale, and dependent on continuous attention. In many settings, teams react only after congestion appears instead of anticipating it with measurable indicators.

This project addresses that gap by turning camera streams into decision-oriented telemetry. The target output includes people in zone, arrival rate, service rate, adaptive wait-time estimates, and alert states for monitoring and automation layers.

1.3.2 Problem Statement

Current operational practices and basic counting tools still have recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they provide weak visibility into queue stability during sudden demand spikes,
- they are often disconnected from automated response workflows.

As a result, interventions such as staff reallocation, lane opening, or escalation can be delayed or triggered on incomplete signals. The problem is both analytical and integration-related, and Table 1.1 maps these pain points to system-level requirements.

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between staff members, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
Queue behavior under load spikes is hard to interpret.	Teams may overreact or underreact when service and arrivals fluctuate quickly.	Adaptive wait-time formula that responds gracefully to changing queue conditions.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

Table 1.1: From operational pain points to system-level requirements

1.3.3 Study of Existing Approaches

Several computer-vision frameworks provide the basic building blocks for person detection and tracking. In parallel, queueing theory gives a formal way to describe service-system behavior and relate arrival rate, service rate, and expected waiting time [1].

In practice, existing commercial and open-source tools cover only parts of the queue-management problem. They fall into three broad families: queue management platforms, sensor-based analytics, and vision-based monitoring frameworks.

1.3.3.1 Queue Management Platforms

Platforms such as Waitwhile [3] and Qminder [4] focus on active queue management. They let customers join virtual queues through kiosks, QR codes, or web links, and they notify staff through SMS or dashboards. Their strength is customer-facing experience: reduced perceived waiting, appointment scheduling, and service routing. However,

they require explicit customer interaction (check-in) and do not observe the physical queue directly. They cannot estimate wait time from the scene itself, and they do not detect congestion unless a customer has already registered.

1.3.3.2 Sensor-Based Analytics

RetailNext [5] and Xovis [6] represent the sensor-based analytics family. RetailNext uses ceiling-mounted AI sensors to track foot traffic, measure conversion rates, and optimize staffing in retail environments. Xovis uses 3D optical sensors for precise people flow and queue length measurement, particularly in airports and large retail spaces. Both provide high-accuracy occupancy and dwell-time data, but they require proprietary hardware installations and are primarily designed for large-scale enterprise deployments. They observe the queue passively but do not typically integrate with external automation workflows or provide open APIs for custom alert routing.

1.3.3.3 Vision-Based Monitoring Frameworks

Open-source tools built on YOLO detection and tracking, such as those provided by Ultralytics [2] and Roboflow [7], offer the computer-vision building blocks needed for queue monitoring: person detection, tracking, and zone-based counting. These frameworks are flexible and can be adapted to specific use cases. However, they typically stop at detection and counting. They do not include queue-theoretic estimation of wait time, stability classification, or integration with notification workflows. The gap between raw detection output and actionable operational metrics is left to the developer.

1.3.3.4 Positioning of the Proposed Solution

The proposed solution is positioned against the main existing approach families in Table 1.2.

Approach family	Strengths	Typical limitation
Queue management platforms (Waitwhile, Qminder)	Customer-facing experience, virtual queues, appointment scheduling.	Require explicit check-in; do not observe physical queues or estimate wait time from the scene.
Sensor-based analytics (RetailNext, Xovis)	High-accuracy occupancy and flow data from dedicated 3D sensors.	Proprietary hardware; limited custom automation; enterprise-scale pricing.
Vision-based detection frameworks (Ultralytics, Roboflow)	Flexible open-source detection and tracking building blocks.	Stop at counting; no queue-theoretic estimation, stability classification, or alert integration.
Manual or occupancy-only monitoring	Easy to deploy and interpret.	Weak predictive value and high subjectivity.
Proposed integrated pipeline	Links perception, queue analytics, stability handling, web monitoring, and automation in one modular architecture.	Requires careful integration and validation across multiple technical domains.

Table 1.2: Positioning of the proposed solution against existing approaches

The proposed solution combines open-source vision-based detection with queue-theoretic analytics, a web monitoring dashboard, and automated alert routing in one modular pipeline without proprietary hardware or explicit customer interaction.

1.3.4 Proposed Solution and Project Objectives

The proposed system integrates perception, analytics, and integration layers in a modular architecture designed to bridge the gap between raw visual observations and actionable operational metrics. Rather than treating these domains in isolation, the architecture maps directly from camera feeds to verifiable queue-state information and automated responses.

To establish clear traceability between the technical solution and measurable project outcomes, the objectives are mapped to delivery modules and observable outputs in Table 1.3.

Objective ID	Objective	Primary module	Observable output
OBJ-01	Real-time detection and tracking	Module A	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Module A	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Module A	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Adaptive wait-time estimation	Module A	Compute wait time with adaptive logic that responds to queue load.
OBJ-05	Web monitoring	Module B	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Module C	Webhook-based alert routing through n8n and Telegram.

Table 1.3: Objective-to-module traceability

1.3.5 Project Scope, Boundaries, and Stakeholders

The implementation scope addresses three distinct but interdependent technical domains: backend perception and analytics modules, API and runtime integration, and web-based operational interfaces. The report presentation mirrors this structure to maintain alignment between architectural decisions and validation activities.

The system is designed for three stakeholder groups with complementary operational responsibilities. Administrators manage account governance and access controls. Managers configure operational settings, monitor live queue status, respond to alerts, and use performance indicators for staffing and service-quality decisions. End users benefit indirectly through reduced waiting times and improved service flow. This separation reflects both organizational hierarchy and the varying levels of technical

abstraction required by each stakeholder group, from per-feed real-time events to aggregated performance trends.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected to deliver functional increments quickly while reducing integration risks early. The project combines multiple technical domains, including computer vision, queue analytics, backend APIs, frontend workflows, and automation, and each one has uncertainties that are difficult to resolve in a single delivery cycle.

Iterative delivery made it possible to validate contracts progressively: first perception outputs, then analytical reliability, then API/runtime integration, then web usability, and finally workflow automation and hardening.

1.4.2 SCRUM

Development followed a sprint-based Scrum methodology with iterative validation cycles. Each sprint focused on a coherent architectural or functional segment, progressing from foundational perception and analytics components through web interface development and finally to automation and system hardening. For report readability and implementation traceability, the work is presented through three logical modules that correspond to the architectural layers and their validation perspectives, as illustrated in Figure 1.2.

The modular organization is summarized in Table 1.4, which maps each module to its constituent sprints, primary focus areas, and validation perspectives.

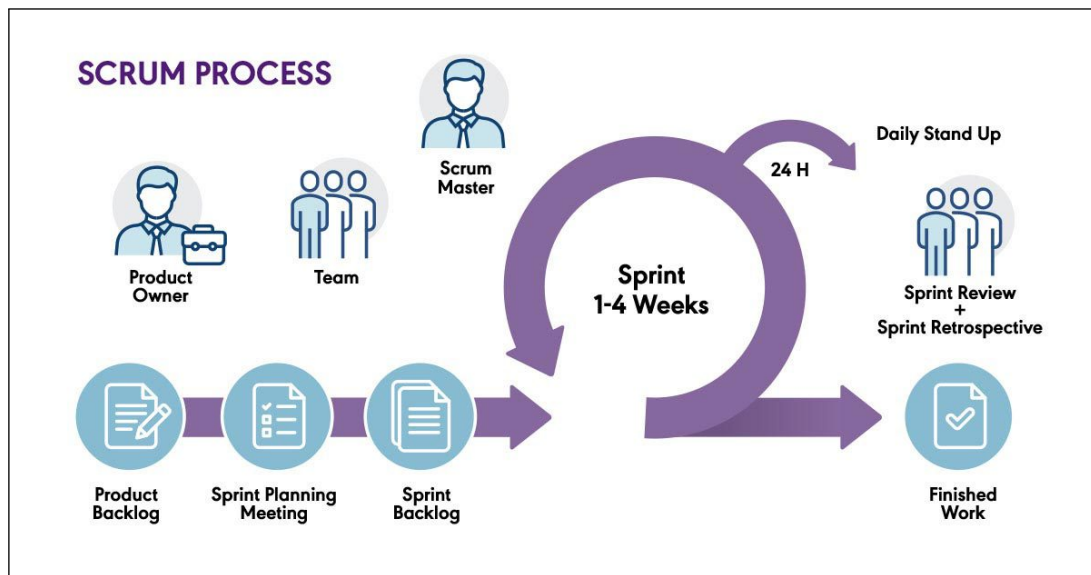


Figure 1.2: Scrum process used to organize the project.

Module	Sprints	Main focus	Validation perspective
Module A: Backend Core	1 to 2	Perception pipeline, queue analytics, camera connectivity, API services, live transport	Perception correctness, metric contracts, and API reliability.
Module B: Web Experience	3	Frontend design system, dashboard scaffolding, live integration, monitoring workflows	Usability, live-rendering stability, and role-based navigation.
Module C: Automation and Finalization	4	n8n alerting, Telegram notifications, evaluation, optimization, and fault tolerance	End-to-end operability and delivery readiness.

Table 1.4: Scrum module mapping used in this report

1.4.3 Sprint Planning and Execution

The project was structured into four iterative sprints, each with defined objectives, deliverables, and validation checkpoints. The sprint plan is summarized in Table 1.5, which details the timeline, focus areas, and traceability to user story implementations

for each sprint.

Sprint	Objectives	Start Date	End Date	User Story IDs
Sprint 1	Perception and queue analytics, detection, tracking, and zone filtering, wait-time estimation and stability handling	16/02/2026	14/03/2026	US1.1–US1.9
Sprint 2	Camera connectivity and discovery, backend API exposure, live transport and feed management	15/03/2026	11/04/2026	US2.1–US2.4
Sprint 3	Web design system and frontend scaffolding, live backend integration, dashboard monitoring and role-based navigation	12/04/2026	09/05/2026	US3.1–US3.9
Sprint 4	n8n automation and Telegram alerting, testing and verification, runtime tuning and fault tolerance	10/05/2026	06/06/2026	US4.1–US4.7

Table 1.5: Sprint planning used in the report

The detailed user stories for each sprint are presented in the backlog tables in Chapters 3 to 6.

1.5 Conclusion

This chapter set the project context and justified the selected delivery strategy. Chapter 2 builds on this baseline by defining the requirements, architecture constraints, and analysis artifacts used to guide implementation decisions.

Table of Content

2.1	Introduction	14
2.2	Requirements Specification	14
2.2.1	Identification of Actors	14
2.2.2	Functional Requirements	15
2.2.3	Non-functional Requirements	17
2.3	Product Backlog and Sprint Organization	17
2.4	Global Use Case Diagram	18
2.5	Project Environment	19
2.5.1	Hardware and Software Environment	19
2.5.2	Development Tools	19
2.5.3	Technologies Used	20
2.6	Software Architecture	21
2.6.1	Overall Architecture	21
2.6.2	Backend Architecture	23
2.6.3	Frontend Architecture	23
2.7	Conclusion	24

2.1 Introduction

This chapter turns the project objective into the requirements that guided the implementation. It identifies the actors, lists the functions and quality constraints they need, and shows how those requirements are spread across the implementation plan.

The requirements are tied to the queue-monitoring pipeline used throughout the project: video ingestion, person detection and tracking, queue-metric estimation, dashboard delivery, authentication, and alert automation.

2.2 Requirements Specification

2.2.1 Identification of Actors

The platform involves two in-system actors and one external operational actor. This separation clarifies which actions are executed inside the software and which actions happen on site.

The actors are:

- **Administrator:** manages manager accounts (create, update, delete) and controls access governance.
- **Manager:** configures sources and thresholds, monitors live queues, reacts to alerts, and reviews performance indicators.
- **On-site Operator (external actor):** receives queue alert notifications in a shared Telegram group and applies field actions, such as opening a new queue line or redirecting customer flow. In the MVP, the operator confirms completion manually in Telegram, and this confirmation is not automatically captured by the application.

This split makes the functional scope easier to explain and validate: access-control,

monitoring, and alert notification actions remain in-system, while physical queue intervention and manual completion confirmation are explicitly modeled as external operational actions.

2.2.2 Functional Requirements

At a high level, the system captures a feed, counts people, estimates queue behavior, updates the dashboard, and sends alerts when needed. These requirements define the minimum end-to-end behavior expected from the platform and are formalized in the traceability matrix shown in Table 2.1.

REQUIREMENTS SPECIFICATION AND ANALYSIS

ID	Requirement	Priority	Verification
FR-01	Detect people in live or recorded streams in near real time.	High	Validate detections and overlays in the perception layer.
FR-02	Preserve identity continuity across frames for queue analysis.	High	Validate tracker ID continuity in runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate polygon inclusion in the zone-management layer.
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in analytics.
FR-05	Compute wait time with adaptive logic that switches between precise and conservative estimates.	High	Validate formula branches and fallback behavior.
FR-06	Expose feed control and live updates through API and websocket channels.	High	Validate endpoint and runtime behavior in API tests.
FR-07	Dispatch alert payloads to automation workflows.	Medium	Validate webhook dispatch and n8n branching.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate persistence behavior in runtime and database checks.
FR-09	Allow manager registration, sign-in, refresh, and logout through secure cookies.	High	Validate register, login, refresh, logout, and session checks.
FR-10	Allow administrator to create, update, deactivate, and reset manager accounts.	High	Validate account-governance endpoints and audit events.
FR-11	Enforce password policy, lockout after repeated failures, and role-based protection on operational endpoints.	High	Validate failed-login counters, lockout behavior, and unauthorized access.
FR-12	Record authentication and account-governance actions for traceability.	Medium	Validate audit-log entries during register and login flows.
FR-13	Protect administrative and operational APIs with role-based access control.	High	Validate unauthorized access paths and audit events.

Table 2.1: Functional requirement traceability matrix

2.2.3 Non-functional Requirements

In addition to functional behavior, the platform must satisfy quality constraints that keep it usable in a live monitoring environment. Because the project combines computer vision, analytics, a web dashboard, and automation workflows, performance and reliability matter as much as correctness. These quality attributes are treated as engineering contracts rather than only design intentions, and the corresponding non-functional requirement quality matrix is summarized in Table 2.2.

ID	Quality	Target indicator	Validation
NFR-01	Performance	Keep the loop near real time and limit alert delay.	Runtime observation under representative feed scenarios.
NFR-02	Scalability	Support independent control for multiple feed contexts.	Feed registry behavior and state-transition checks.
NFR-03	Reliability	Recover gracefully from camera drops and webhook failures.	Reconnect paths and retry/fallback test scenarios.
NFR-04	Usability	Keep setup and monitoring clear for non-developer users.	Dashboard integration tests and workflow walkthroughs.
NFR-05	Security	Restrict integration access through secrets and credentials.	Webhook secret validation and credential checks.
NFR-06	Maintainability	Keep the architecture modular and testable.	Code review and test-suite coverage by module.

Table 2.2: Non-functional requirement quality matrix

2.3 Product Backlog and Sprint Organization

The product backlog was prioritized according to business value and technical dependency and organized into four implementation sprints. To manage complexity and validate incremental progress, the work was structured around the system’s data flow

pipeline: from perception and analytics foundations, through web-based monitoring, to automation and evaluation. The first sprints focused on backend foundations because the dashboard and automation layers depend on a stable metric contract. Each subsequent sprint added layers of functionality while maintaining testability and integration with earlier work.

2.4 Global Use Case Diagram

The global use case diagram gives a high-level view of how the administrator and manager interact with the platform during configuration, monitoring, and alert handling, as shown in Figure 2.1.

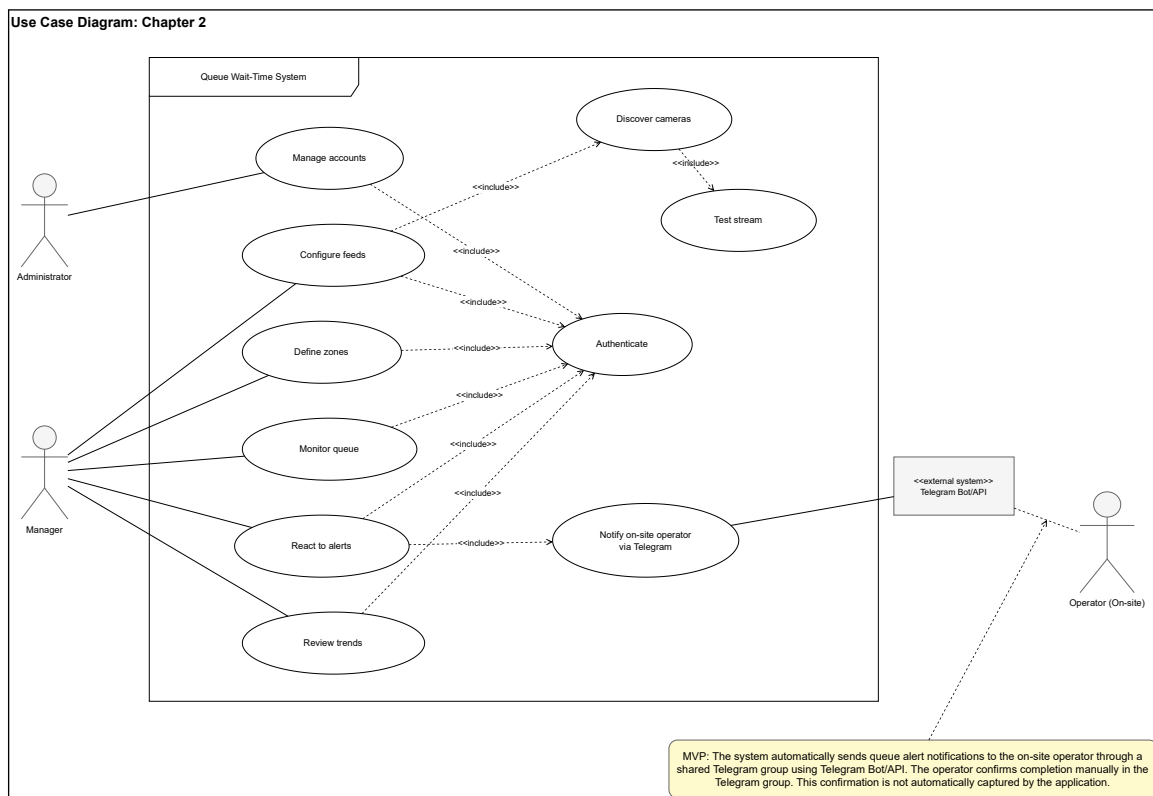


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At the system level, the use cases can be grouped into four interactions: configuration, live monitoring, analytics interpretation, and alert handling/resolution, with the main actor responsible for each use case listed in Table 2.3.

Use case ID	Use case summary	Primary actor
UC-01	Create, update, or remove manager accounts.	Administrator
UC-02	Register and configure a video feed source.	Manager
UC-03	Define or update queue zone geometry.	Manager
UC-04	Monitor live queue metrics and feed status.	Manager
UC-05	Receive and interpret warning or critical alerts.	Manager
UC-06	Automatically notify the on-site operator via Telegram when an alert is detected.	System automatic behavior
UC-07	Review trends and performance indicators.	Manager

Table 2.3: Primary global use cases

2.5 Project Environment

2.5.1 Hardware and Software Environment

The hardware environment used during development is shown in Table 2.4.

Component	Type
Processor	Intel Core i5 12th Generation
Memory (RAM)	16 GB
Graphics card	NVIDIA GeForce RTX 4060
Storage	512 GB SSD

Table 2.4: Hardware environment

2.5.2 Development Tools

The project also relied on a small set of development tools for editing, versioning, diagramming, container support, and report generation. Keeping them separate from

the technology stack makes it easier to distinguish what was used to build the project from what the project actually runs on, as listed in Table 2.5.

Tool	Role in the project	Typical use
VS Code [8]	Main editor for the backend, frontend, and report sources.	Day-to-day implementation, review, and navigation.
Git / GitHub [9], [10]	Version control and change history tracking.	Commit management, branching, and collaboration.
draw.io [11]	Diagram authoring for architecture and UML models.	Use case, UML class, and system architecture diagrams.
Docker / Docker Compose [12]	Local service orchestration and reproducible setup.	Supporting services such as MediaMTX and n8n.

Table 2.5: Development tools used in the project

2.5.3 Technologies Used

The technology stack follows the main stages of the system, as summarized in Table 2.6.

Layer	Main technologies	Role in the project
Vision pipeline	YOLO26, ByteTrack, and OpenCV [2], [13], [14]	Detects people, tracks them across frames, and prepares the visual input for queue analysis.
Queue analytics	NumPy, Pandas, SciPy, and M/M/1 modeling [1], [15], [16], [17]	Computes arrival rate, service rate, and wait time.
Backend API	FastAPI and Pydantic v2 [18], [19]	Exposes typed endpoints, validates payloads, and coordinates the backend services.
Live transport	WebRTC for IP cameras, MediaMTX WHEP gateway, and MJPEG playback for local MP4 videos [20], [21]	Delivers low-latency video streaming for camera feeds while keeping a simpler HTTP playback path for uploaded videos.
Frontend	React, TypeScript, Vite, and Tailwind CSS [22], [23], [24], [25]	Implements the dashboard, user interactions, and responsive presentation layer.
Automation	n8n workflows and Telegram notifications [26], [27]	Routes alerts and sends operational messages to users.
Persistence	SQLite [28]	Stores metrics, metadata, and alert history for later analysis.

Table 2.6: Technologies used in the project

This selection keeps the implementation modular from perception through automation.

2.6 Software Architecture

2.6.1 Overall Architecture

The architecture is described as four stages that match the flow of data through the system, from video acquisition to alert delivery, as illustrated in Figure 2.2.

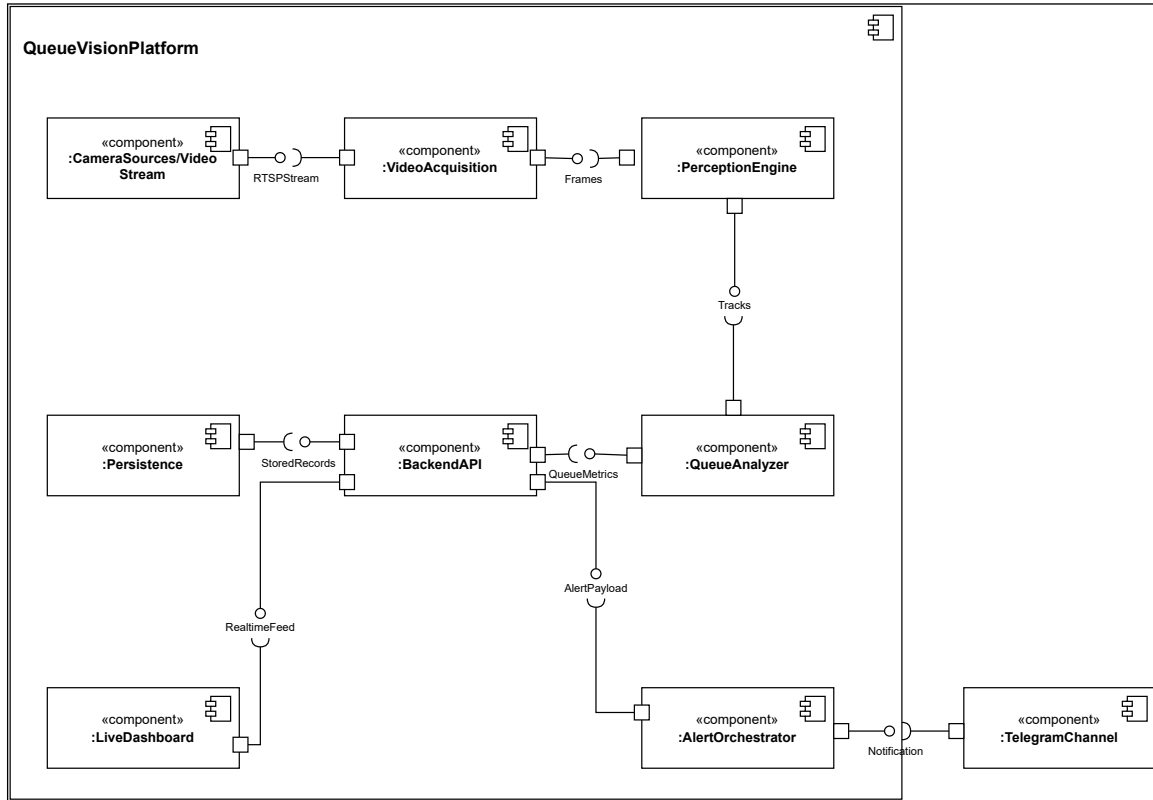


Figure 2.2: Overall system architecture from video acquisition to alert delivery.

1. **Video acquisition:** camera sources and video streams (RTSP, ONVIF, and local files) feed into the video acquisition component, which handles frame capture and stream management [29], [30].
2. **Perception:** the perception engine runs detection and tracking on each captured frame and exposes tracked identities through a defined interface to the queue analyzer.
3. **Analysis:** the queue analyzer receives tracked events and computes queue metrics (λ, μ, W) using adaptive wait-time estimation logic.
4. **Output and integration:** the backend API acts as the hub that receives queue metrics and distributes them across three outputs — the live dashboard via a real-time feed, the persistence layer (SQLite) for historical records, and the alert orchestrator (n8n) which routes alert payloads to the Telegram notification channel.

This separation keeps the four stages independently testable. Queue formulas in

the analyzer depend only on perception events; the backend API depends only on the metric contract; and the dashboard, persistence, and alert orchestration are each driven by the API without depending on each other.

2.6.2 Backend Architecture

The backend exposes REST and websocket interfaces for feed lifecycle control and live metrics. It includes modules for detection orchestration, tracking, queue analysis, threshold evaluation, webhook delivery, and camera-source management.

Feed-level orchestration lets each camera be started, stopped, or refreshed independently. If one feed fails, the others keep running, and the dashboard still receives a predictable event stream.

We treat persistence and integration as core responsibilities rather than add-ons: a durable backend state store preserves operational context, while webhook delivery turns queue events into external notifications.

2.6.3 Frontend Architecture

The frontend is built as a component-based interface with typed API integration, feed-oriented dashboard composition, and websocket-driven synchronization. Data access is isolated in hooks and client utilities, while pages and reusable components focus on presentation and interaction.

That separation keeps API changes localized to the integration layer and lets the visual components stay stable even when the backend contract changes. It also makes the dashboard easier to extend when new feed controls or indicators are added.

2.7 Conclusion

This chapter establishes the functional baseline, the quality constraints, and the architecture choices that guide the rest of the report. The next chapters turn these requirements into perception, connectivity, automation, and evaluation work.

Sprint 1: Perception and Queue Analytics

Table of Content

3.1	Introduction	26
3.2	Sprint Objectives and Backlog	26
3.3	Sprint Backlog	26
3.4	Design	28
3.4.1	Perception and Analytics Flow	28
3.5	Implementation	30
3.5.1	Perception Pipeline	30
3.5.2	Queue Event Windows and Rate Estimation	32
3.5.3	Wait-Time Logic and Adaptive Estimation	32
3.5.4	Runtime Metric Contract	33
3.6	Conclusion	33

3.1 Introduction

Sprint 1 built the full path from raw video to actionable queue indicators. It started by detecting people, tracking them across frames, and filtering observations to the queue zone. Once that perception baseline was reliable, the sprint moved to queue interpretation: estimating how fast people arrive, how quickly service handles them, and how long a new arrival should expect to wait.

Perception and analytics were delivered together so that the later sprints could reuse a stable metric pipeline instead of rebuilding it from raw counts.

3.2 Sprint Objectives and Backlog

This sprint was focused on delivering two complementary layers: a reliable perception baseline for tracking people across frames within designated queue zones, and a queue analytics layer that converts tracked observations into operational metrics. The specific objectives are captured in the sprint backlog, which separates manager-facing outcomes from technical implementation concerns.

3.3 Sprint Backlog

The Sprint 1 backlog separates manager-facing user stories from technical implementation tasks. User stories represent stakeholder needs and value; technical tasks are internal infrastructure concerns. This structure ensures that traceability focuses on business outcomes while documenting all delivery work. The user stories in Sprint 1 represent manager needs for reliable perception and analytics, as shown in Table 3.1.

SPRINT 1: PERCEPTION AND QUEUE ANALYTICS

User Story ID	User Story	Task ID	Task
US1.1	As a manager, I want to detect people in the video stream so that queue observations start from a reliable signal.	T1.1	Configure the detector to identify people in live and recorded video.
US1.2	As a manager, I want to keep the same person identifiable across frames so that counts remain consistent.	T1.2	Integrate tracking to preserve identity across consecutive frames.
US1.3	As a manager, I want to restrict counting to queue zones so that irrelevant movement does not affect the queue state.	T1.3	Define queue zones as polygons and filter detections by zone.
US1.6	As a manager, I want queue rates estimated from tracked people so that the dashboard can interpret flow intensity.	T1.6	Group tracked events into short windows and compute arrival and service rates.
US1.7	As a manager, I want wait time computed adaptively so that the result remains useful in real conditions.	T1.7	Apply the M/M/1 formula when arrivals are below service capacity and fallback to occupancy-based estimation otherwise.
US1.9	As a manager, I want a clear metric output so that the dashboard can reuse these values without reprocessing them.	T1.9	Publish rate estimates, wait time, and people-in-zone occupancy.

Table 3.1: Sprint 1 user stories

Supporting technical tasks address internal implementation requirements that enable these user stories, as detailed in Table 3.2.

Task ID	Task	Subtask ID	Subtask
T1.4	Connect capture, detection, tracking, zone filtering, and display into one continuous flow.		Implement runtime orchestration that integrates all perception components.
T1.5	Expose runtime settings so that the pipeline can be adapted without code changes.		Provide configurable parameters for frame handling and visualization.
T1.8	Add guards for empty windows, low counts, and bounded fallback output to protect calculations from division-by-zero and noise.		Implement defensive bounds checking in the analytics layer.

Table 3.2: Sprint 1 technical tasks

3.4 Design

3.4.1 Perception and Analytics Flow

The Sprint 1 workflow from frame ingestion to queue metrics output occurs at two levels: the overall perception pipeline and the detailed queue analytics transformation. The design must address several runtime conditions and operational scenarios that arise during real-world queue monitoring.

At the perception level, the system handles three key scenarios: when a person enters the queue zone, they are treated as a possible arrival; when the same person appears in consecutive frames, the system maintains stable identification to avoid double-counting; and when people appear outside the zone, they are filtered out to keep measurements focused on the queue area.

At the queue-operation level, the system must respond gracefully to three distinct conditions. In a normal queue, arrivals remain below service capacity, allowing the M/M/1 formula to provide precise rate-based estimates. When arrivals approach

service capacity, the system seamlessly switches to a conservative occupancy-based estimate. When no service events are available, the output remains bounded and safe for downstream processing. This adaptive logic ensures that a single metric responds correctly to real-world conditions without requiring separate state signals.

At the high level, Figure 3.1 shows the end-to-end flow from a video source (local file or live camera stream) through detection, tracking, zone filtering, and finally to queue analysis that computes arrival rate, service rate, and expected wait time.

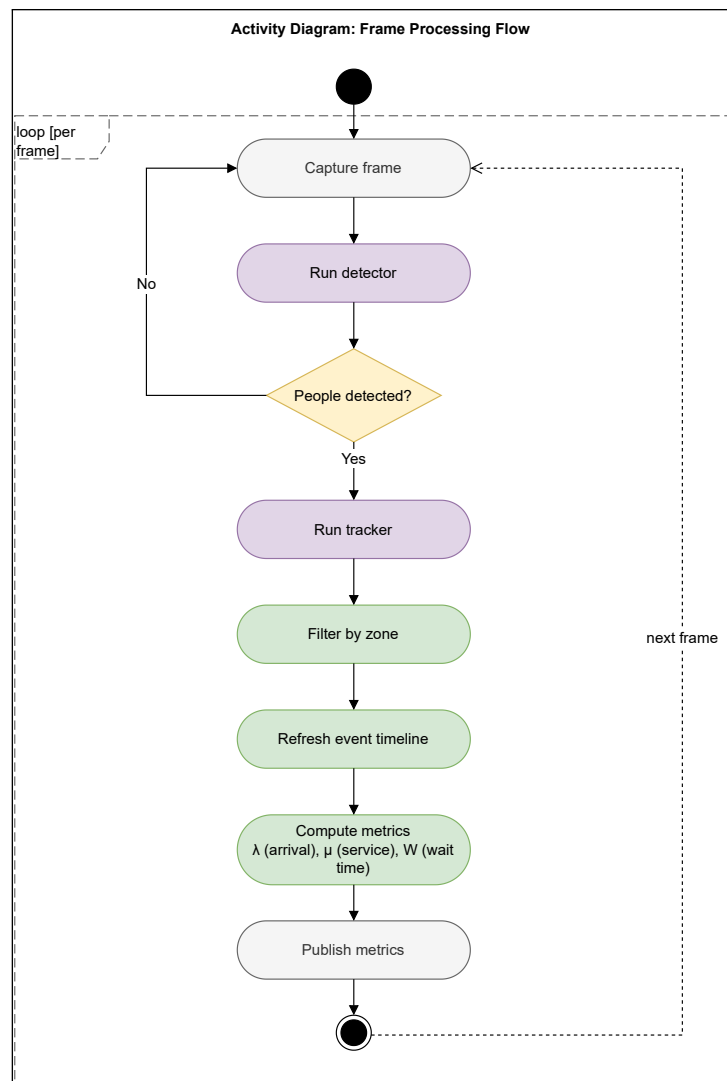


Figure 3.1: Sprint 1 activity flow from frame ingestion to queue metrics output.

At the detailed level, Figure 3.2 focuses on the queue analytics transformation that converts tracked people inside the zone into the metric payload consumed by the dashboard and later automation layers. The analytics step compares current tracked

identifiers with the previous observation window to identify arrivals and departures. These events are stored in short windows, then converted into rates (λ and μ). Before applying the wait-time formula, the system checks whether enough events were observed, whether μ is positive, and whether arrivals remain below service capacity. If these guards are satisfied, it uses the M/M/1 expression; otherwise, it falls back to a bounded occupancy-based estimate.

3.5 Implementation

3.5.1 Perception Pipeline

The perception pipeline integrates three core components: detection, tracking, and zone filtering. The detection layer identifies people in each frame using YOLO and is tuned to balance speed and accuracy, ensuring the system remains responsive without losing critical observations. The detection output is then fed to the tracking layer, which preserves person identity across consecutive frames using ByteTrack. This identity continuity is essential for reliable arrival and departure counting, as it prevents double-counting from single-frame detections alone. Finally, zone-aware filtering restricts all measurements to a polygon-defined queue area, eliminating unrelated movement elsewhere in the scene and ensuring the metric reflects only the relevant part of the frame.

The pipeline integration is orchestrated in the runtime layer, which supports handling diverse input sources (local files, webcams, and live camera streams), adjusting frame-processing frequency to maintain system responsiveness, rendering overlays to display detections, tracker identities, queue zones, and computed metrics, and operating without visual output when needed for headless integration with downstream systems.

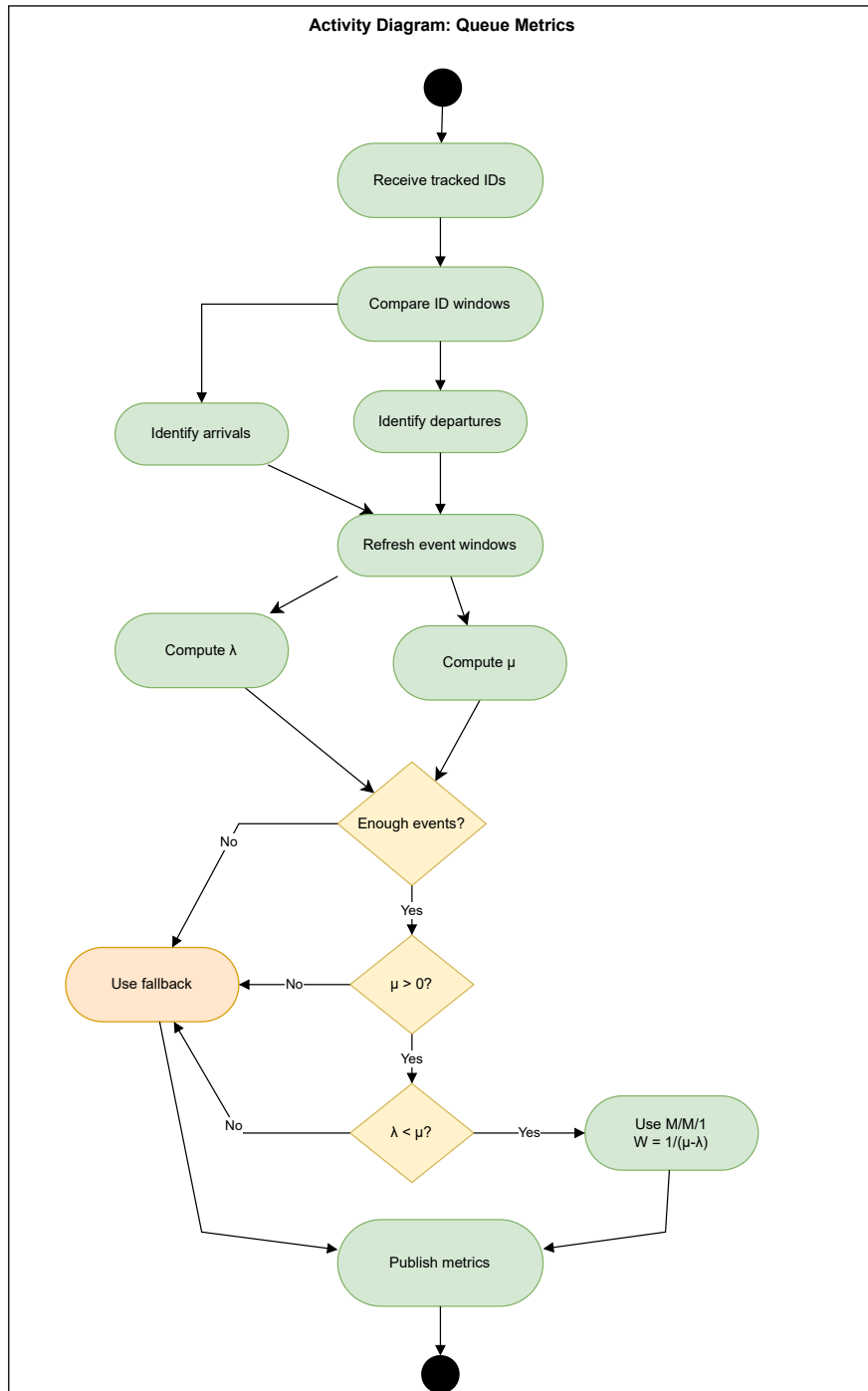


Figure 3.2: Sprint 1 analytics activity diagram for queue metric generation.

3.5.2 Queue Event Windows and Rate Estimation

The queue-analysis step uses short time windows to count arrivals and departures. Let N_a and N_d be the number of events observed in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

From these counts, the system estimates the arrival rate and the service rate. To avoid unstable values at the beginning of a run, the implementation waits until enough events have been observed before returning non-zero rates.

3.5.3 Wait-Time Logic and Adaptive Estimation

The system estimates wait time using an adaptive formula that responds to queue conditions:

$$W = \begin{cases} \frac{1}{\mu - \lambda} & \text{if } \lambda < \mu \text{ and } \mu > 0 \\ \frac{L}{\mu} & \text{otherwise} \end{cases}$$

where L is the current number of people in the zone. We use the M/M/1 sojourn-time expression because the project models each monitored lane as a single service channel with one effective queue, which matches the cashier-style setting studied in the report. Other models were less appropriate here: M/M/c would assume multiple parallel servers, which is not the default lane setup, and Little's Law is useful for validation but does not directly produce a forward wait-time estimate from the instantaneous rates alone.

The adaptive formula ensures the output always remains safe and interpretable: when arrivals are below service capacity, the formula is precise; when arrivals exceed capacity, it gracefully degrades to a conservative estimate based on current queue size. When no service events are observed, the implementation returns zero.

3.5.4 Runtime Metric Contract

The final metric output includes:

- point estimates (λ , μ , W),
- people-in-zone occupancy.

These values are then reused by logs, overlays, webhook payloads, and dashboard events [1], [14].

3.6 Conclusion

Sprint 1 delivered the complete path from raw video to operational queue indicators, establishing a trustworthy metric layer for downstream systems. The integration of detection, tracking, and zone filtering produced a reliable perception baseline, while the queue analytics layer converted tracked observations into arrival rates, service rates, and adaptive wait-time estimates.

The key technical achievement was the adaptive wait-time formula, which remains interpretable and safe across all operating conditions: it provides precise estimates when arrivals are below service capacity, gracefully degrades to occupancy-based estimates when the queue grows, and returns bounded values when service events are unavailable. This design ensures that downstream layers—the dashboard, persistence, and automation—can operate on structured and stable signals rather than raw detection counts.

By connecting perception to interpretation in a single delivery, the sprint reduced false continuity problems through tracking and zone logic, while the queue model provided the necessary foundation for later integration with web monitoring and automated alerting workflows. The metric contract established in this sprint is designed to be reused consistently throughout the system's remaining layers.

Table of Content

4.1	Introduction	35
4.2	Sprint Objectives and Backlog	35
4.3	Design	36
4.3.1	Backend Domain Model and Class Structure	36
4.4	Implementation	37
4.4.1	Source Integration and Camera Discovery	37
4.4.2	API and Live Transport	38
4.4.3	Persistence, State Recovery, and System Resilience	41
4.5	Conclusion	41

4.1 Introduction

Sprint 2 expanded the project beyond a single local pipeline to support multiple camera sources. The backend was restructured to manage camera onboarding, feed control, and live delivery, providing a structured API that the frontend could depend on.

4.2 Sprint Objectives and Backlog

Sprint 2 transformed the backend from a single-pipeline script into a structured service layer capable of managing multiple camera sources, providing a reliable API for feed lifecycle operations, and delivering live streams with fallback awareness. The sprint objectives are captured in the user stories shown in Table 4.1, which outline manager needs for secure camera connectivity, automatic discovery, centralized feed management, and reliable live delivery.

User Story ID	User Story	Task ID	Task
US2.1	As a manager, I want IP camera streams connected securely so that live sources can be reused reliably.	T2.1	Add authentication, transport control, reconnection, and snapshots for RTSP feeds.
US2.2	As a manager, I want cameras discovered automatically so that stream addresses do not need to be hardcoded.	T2.2	Detect IP cameras and resolve their stream addresses.
US2.3	As a manager, I want feed management exposed through the API so that camera lifecycle actions are centralized.	T2.3	Expose feed control, metadata handling, and diagnostic operations.
US2.4	As a manager, I want live delivery to stay available so that the dashboard can keep showing the active feed.	T2.4	Support live delivery through real-time events and a layered video transport strategy.

Table 4.1: Sprint 2 user stories

4.3 Design

4.3.1 Backend Domain Model and Class Structure

The backend architecture organizes data entities hierarchically to support multi-manager and multi-feed scenarios while maintaining consistent state. The model structures manager accounts, feed configurations, video sessions, queue metrics, and alerts as a unified domain, where establishments parent both managers and feed configurations, and managers own feed definitions. This hierarchy ensures proper access scoping and state recovery across system restarts.

The core entities and their relationships are illustrated in Figure 4.1.

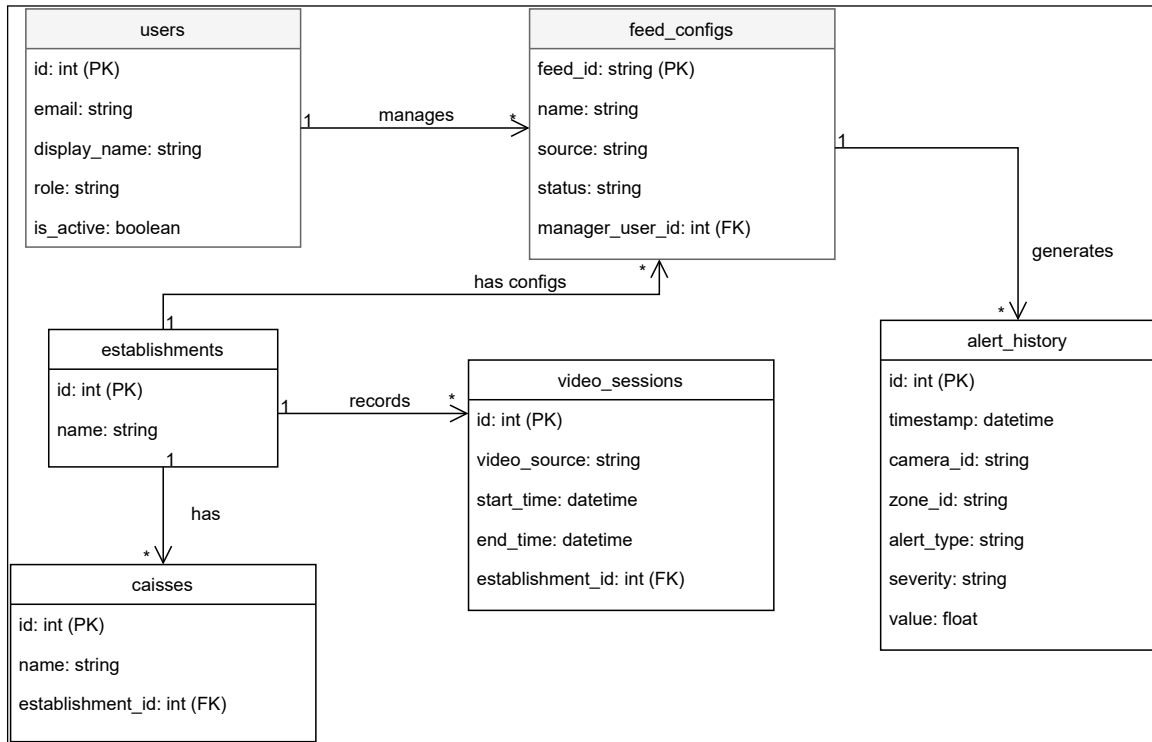


Figure 4.1: Sprint 2 backend class diagram showing core entities for feed orchestration and persistence.

4.4 Implementation

4.4.1 Source Integration and Camera Discovery

The source integration layer supports multiple input types and network scenarios through a unified abstraction. IP cameras are connected via RTSP with secure credential handling, configurable TCP/UDP transport selection, connection validation, and automatic reconnection on stream interruption. Local files and webcams are routed through separate pathways to reduce complexity in the core source layer.

Camera discovery uses the ONVIF protocol to automatically identify cameras and retrieve stream information, eliminating hardcoded addresses. The implementation sends WS-Discovery probes over multicast UDP, processes ONVIF device responses, resolves media services, and requests RTSP URIs through SOAP calls. Streams are validated with OpenCV before registration to ensure connectivity. The workflow accommodates varying ONVIF implementations through a three-phase protocol: multicast

UDP discovery, direct media service access for cameras that expose it, and device service indirection for cameras requiring capability queries first. This approach ensures robust discovery across heterogeneous camera deployments. The workflow was validated against a Digital Watchdog IP camera on a shared network.

The discovery process is exposed through backend endpoints for probe-based detection, stream resolution, camera testing, and snapshot capture. Credentials are handled securely without log exposure, and transport configuration adapts to stream requirements based on network conditions and source type.

The logical workflow of this discovery process is illustrated in Figure 4.2.

The protocol-level details are captured in Figure 4.3, which illustrates the three-phase ONVIF sequence: multicast UDP discovery, direct media service access, and device service indirection.

4.4.2 API and Live Transport

The backend exposes a structured REST and WebSocket API that coordinates feed management, source onboarding, metadata operations, system integration, and live event delivery. Access control is enforced at the session level: managers interact only with feeds in their account, while administrators maintain broader account governance scope. Access rules apply consistently across all operations including feed listing, creation, lifecycle control, snapshots, and real-time subscriptions.

Transport selection adapts to source type to minimize latency while maintaining fallback capability. IP camera feeds prioritize WebRTC for low-latency delivery, while local MP4 files use MJPEG endpoints. The backend communicates feed readiness, source type, and preferred playback paths to the frontend. For WebRTC-capable IP feeds, the backend facilitates browser offer/response negotiation through the media server. When the preferred transport is unavailable (feed down, service unreachable), the backend reports the condition explicitly, allowing the frontend to switch to fallback

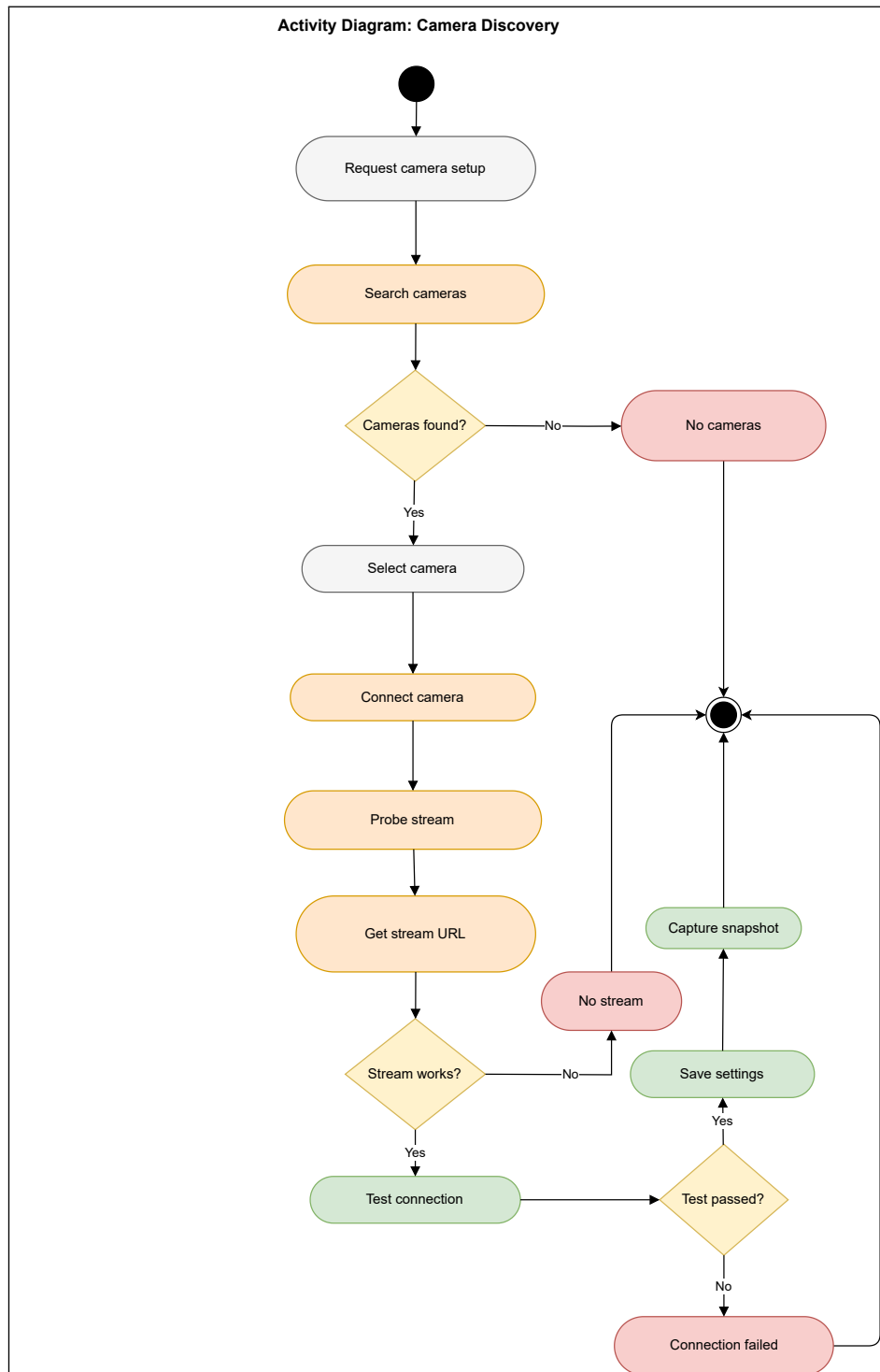


Figure 4.2: Camera discovery activity diagram: ONVIF discovery, RTSP resolution, and stream validation.

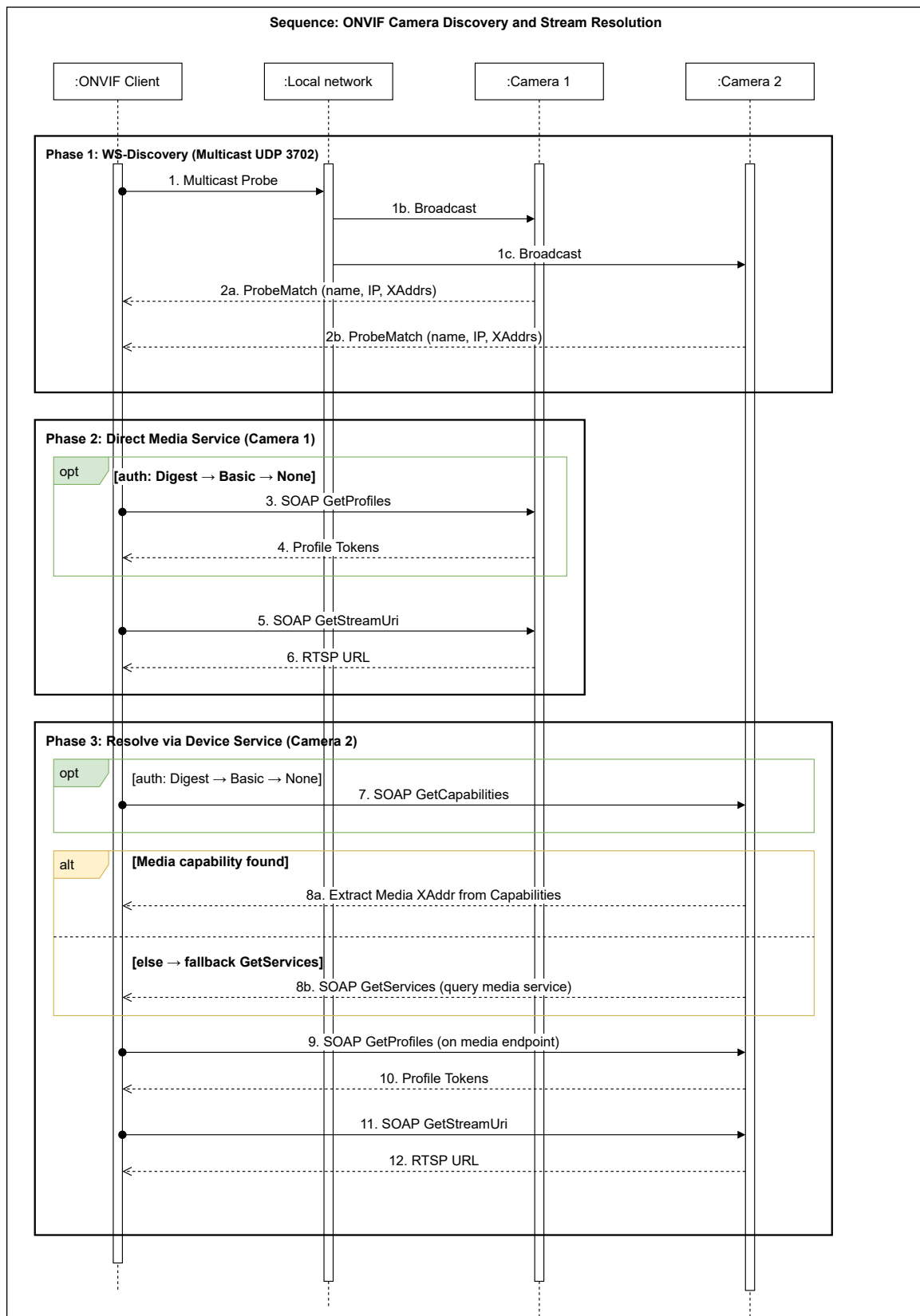


Figure 4.3: ONVIF protocol sequence: WS-Discovery broadcast, direct media service resolution, and device service indirection.

views rather than timing out silently. Transport configuration is externalized through environment variables for media URL, control URL, timeout, and feature flags.

Runtime control coordinates feed state, persistence synchronization, worker lifecycle management, and event dispatch through the live channel, which broadcasts snapshots, feed status changes, metric updates, alerts, and warnings. For local MP4 videos, a shared MJPEG stream ensures viewing continuity during runtime mode transitions.

4.4.3 Persistence, State Recovery, and System Resilience

The persistence layer uses SQLite to maintain establishments, managers, feed configurations, video sessions, alert history, and system state. The schema organizes entities hierarchically—establishments parent both managers and feed configurations; managers own feed definitions; sessions link back to establishments—ensuring proper access scoping and state isolation across restarts.

On startup, the persistence layer reloads saved feed definitions and resets any open sessions to a consistent state. This recovery mechanism prevents orphaned sessions and ensures feeds restore to their last known configuration without state inconsistency. The key architectural insight is that reliable connectivity extends beyond reaching a camera to maintaining application state across stream failures, transport mode switches, and system restarts. By explicitly managing recovery state and fallback conditions, the sprint ensured the backend could tolerate transient failures and resume operation reliably, a requirement critical for operational deployments.

4.5 Conclusion

Sprint 2 transformed the backend into a structured service layer capable of discovering and managing multiple camera sources, maintaining consistent feed state, and delivering live events reliably to the frontend. The sprint addressed three critical challenges:

integrating heterogeneous camera types through ONVIF discovery and RTSP connectivity, maintaining access control and state consistency across multi-manager deployments, and implementing transport fallback strategies that ensure live delivery resilience.

The architectural achievement was recognizing that backend reliability requires more than connectivity—it demands explicit state recovery, transport flexibility, and graceful degradation when preferred transport becomes unavailable. By embedding these capabilities into the core backend design, Sprint 2 established a foundation that enabled straightforward frontend integration and positioned the system for operational deployment.

Table of Content

5.1 Introduction 45

5.2 Sprint Objectives and Backlog 45

5.2.1 User Stories 45

5.2.2 Technical Tasks 47

5.3 Design 47

5.3.1 Visual Design and Interface Architecture 47

5.3.2 Dashboard Data Flow and Integration Architecture 48

5.3.3 Design Rationale 48

5.4 Implementation 50

5.4.1 Project Setup and Layout Skeleton 50

5.4.2 Styling and Reusable Interface Rules 50

5.4.3 API Integration 50

5.4.4 Authentication and Role-Based Navigation 50

5.4.5 Dashboard State Orchestration and Live Update Reliability 51

5.4.6 Live Video Delivery and Transport Selection 53

5.4.7 Dashboard UI Integration 54

5.4.8 Manager Onboarding Workflow 54

5.4.9 Analytics and Historical Reporting 56

5.4.10 Integration Testing 58

5.5	Conclusion	60
-----	----------------------	----

5.1 Introduction

Sprint 3 implemented the frontend from initial design work through live backend integration. The sprint began by establishing the visual design system and page structure, then connected the interface to backend services and runtime events so the dashboard could respond to live data, transport changes, and feed lifecycle actions.

Completing the design foundation first allowed the integration work to focus on behavioral correctness and reliability rather than redesigning the interface during implementation.

5.2 Sprint Objectives and Backlog

Sprint 3 pursued two parallel tracks: establishing a visual design foundation and then integrating the frontend with live backend services. The design phase established the visual language, page structure, and reusable component abstractions before implementation began. The integration phase connected the interface to backend services, ensuring responsiveness across transport state changes and maintaining operational clarity during feed management and fallback conditions. The sprint backlog organized these objectives into manager-facing user stories and technical implementation tasks.

5.2.1 User Stories

Table 5.1 lists the manager and admin requirements for Sprint 3.

SPRINT 3: WEB DASHBOARD AND LIVE MONITORING

User Story ID	User Story	Task ID	Task
US3.1	As a manager, I want a consistent visual style so that the dashboard feels like one coherent product.	T3.1	Define colors, typography, spacing, and reusable component rules.
US3.2	As a manager, I want the main page layouts prepared so that onboarding, monitoring, editing, and analytics screens are available.	T3.2	Prepare the landing page, dashboard, setup wizard, zone editor, analytics, and settings layouts.
US3.3	As an admin, I want to create and maintain manager accounts so that system access is centrally controlled.	T3.3	Build the admin dashboard for manager account creation, editing, and deletion.
US3.4	As a manager, I want the dashboard connected to backend responses so that the interface reflects live operational data.	T3.4	Implement the API integration layer and typed request handling.
US3.5	As a manager, I want to manage feeds from the interface so that feed lifecycle actions stay centralized.	T3.5	Connect login state, protected routes, and feed control actions to the dashboard.
US3.6	As a manager, I want live updates without refreshing so that changes are visible immediately.	T3.6	Add websocket-driven live state updates and dashboard orchestration.
US3.7	As a manager, I want a suitable video transport chosen automatically so that the feed remains usable when conditions change.	T3.7	Add transport selection, fallback streaming, and reconnection logic.
US3.8	As a manager, I want the dashboard to stay usable during interruptions so that monitoring does not stop during transient failures.	T3.8	Add fallback polling and stale-data guards.

Table 5.1: Sprint 3 user stories

5.2.2 Technical Tasks

Table 5.2 lists the internal implementation work for integration and testing.

Task ID	Task	Subtask ID	Subtask
T3.10	Set up the frontend project with shared components and page scaffolding so that the interface can grow without duplication.		Create reusable UI component library and establish project structure.
T3.11	Validate dashboard setup and onboarding flows with integration tests to ensure the integration behaves correctly end to end.		Implement scenario-based tests for dashboard lifecycle and error paths.

Table 5.2: Sprint 3 technical tasks

5.3 Design

5.3.1 Visual Design and Interface Architecture

The design follows a “Terminal Minimal” style optimized for sustained monitoring operations. The visual language emphasizes dark operational surfaces to reduce fatigue during extended use, semantic colors for status and alert severity distinction, card-based information organization with consistent hierarchy, and explicit workflow scaffolding in setup and zone-editing contexts.

The interface is organized around seven primary pages: landing, dashboard, setup wizard, zone editor, analytics, settings, and heatmap. This navigation structure was established early as the scope boundary, ensuring that the integration phase could focus on behavior and reliability rather than simultaneous redesign.

5.3.2 Dashboard Data Flow and Integration Architecture

The dashboard operates through a structured data flow that balances real-time responsiveness with operational resilience. Initial access requires authentication; once authenticated, the dashboard issues REST queries for feed configuration and system health, then opens a persistent WebSocket connection to the backend. The WebSocket broadcasts an initial metrics snapshot, followed by continuous feed lifecycle events, metric updates, alerts, and warnings. Feed control actions are sent as REST commands and return as status updates through the WebSocket channel.

This architecture is designed for operational continuity: when live WebSocket events stop arriving, the dashboard automatically switches to periodic polling, ensuring managers retain visibility even during transient network failures. For video playback, WebRTC delivers low-latency streams for IP cameras through the backend, while MJPEG serves local video files. The fallback strategy ensures that authentication failures, WebSocket drops, or WebRTC unavailability do not blind the operator—the interface can still poll metrics and display fallback video streams.

Figure 5.1 illustrates this complete data flow during setup, runtime operation, and transport negotiation.

5.3.3 Design Rationale

Prioritizing the design phase as a foundation step proved valuable for the integration work that followed. By settling navigation structure, component boundaries, and interaction patterns before connecting live data and transport logic, the team could focus integration efforts on behavioral correctness and reliability rather than simultaneous redesign. This separation of concerns kept the integration phase tractable and prevented design churn from blocking delivery.

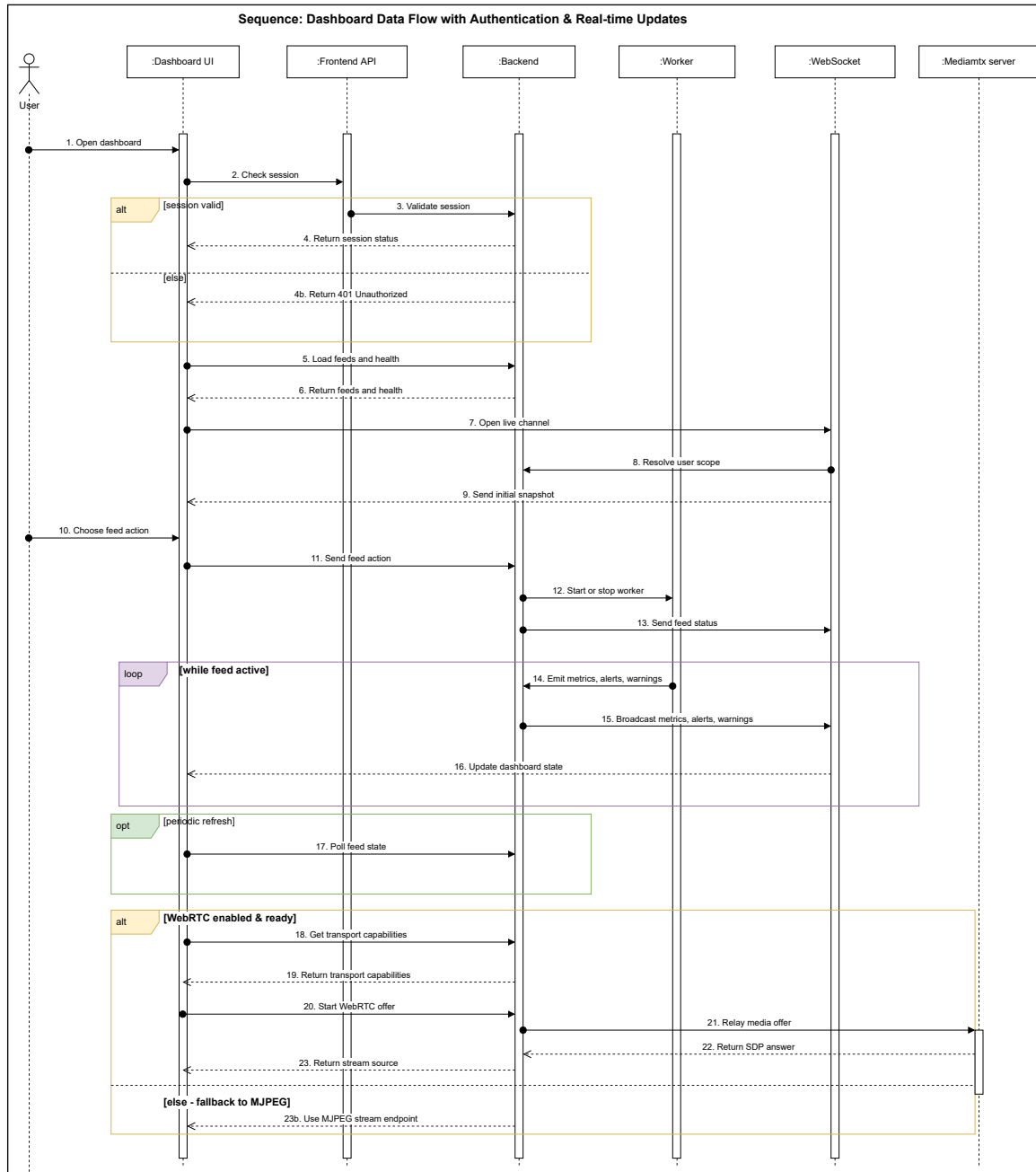


Figure 5.1: Dashboard data flow sequence showing protected access (alt), metric broadcasts (loop), fallback polling (opt), and video transport negotiation (alt).

5.4 Implementation

5.4.1 Project Setup and Layout Skeleton

The design specification was translated into an application skeleton. Setup established the application entry point, protected navigation, the dashboard layout, and shared components across all pages.

Page organization followed the operational workflow: onboarding precedes configuration, which precedes supervision. The skeleton was structured as an operational path rather than a disconnected set of screens.

5.4.2 Styling and Reusable Interface Rules

The visual system was defined once and applied consistently. Spacing, typography, surface treatment, and semantic colors were treated as reusable rules rather than per-page decisions. This consistency prevented design drift: components appearing across multiple pages behave as unified product elements rather than restyled instances.

5.4.3 API Integration

A dedicated API layer structures communication with the backend, defining data types consumed by the dashboard and centralizing error handling. This layer serves as the contract boundary: the UI does not depend on backend implementation details, and feed lists, dashboard summaries, and management actions are explicit typed responses.

5.4.4 Authentication and Role-Based Navigation

Sprint 3 integrated authentication and role-based access control into the dashboard. The auth provider clears cached data on session changes; protected routes redirect users based on their assigned role; administrators access the manager administration

interface while managers reach the monitoring dashboard. This alignment with backend access rules prevents interface mismatch and ensures users only access appropriate functionality.

Three key screens implement this logic: the manager sign-in screen validates credentials before granting dashboard access, the manager account creation screen allows administrators to register new managers, and the administrator interface provides centralized account governance. These screens are shown in Figures 5.2, 5.3, and 5.4. The practical result is that authentication, account management, and role-based routing remain coherent from the user's perspective: each user type encounters only the appropriate interface for their operational role.

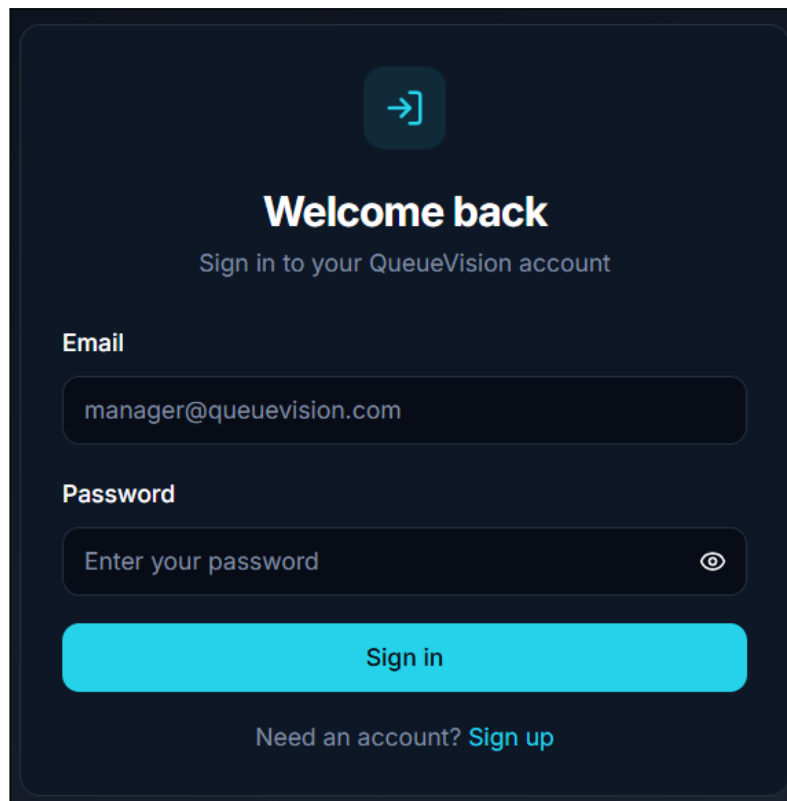
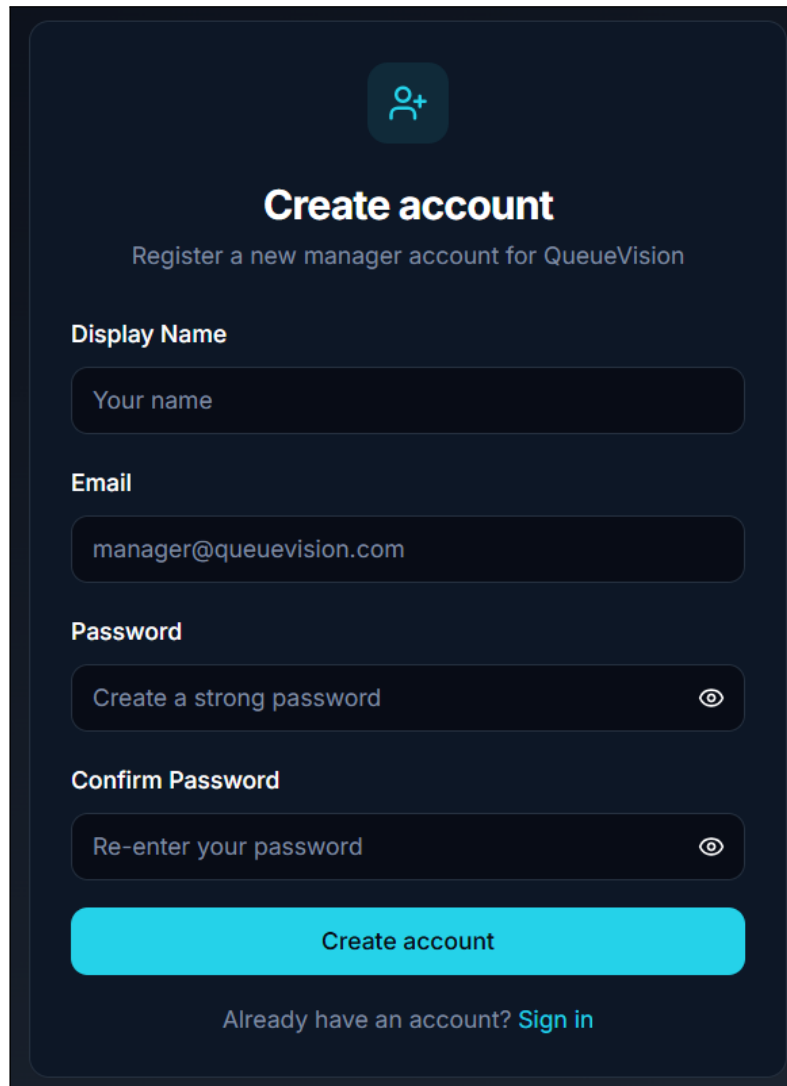


Figure 5.2: Manager sign-in screen for dashboard access.

5.4.5 Dashboard State Orchestration and Live Update Reliability

A state orchestration layer coordinates data fetching and backend updates. From received feed data, the dashboard derives operational indicators: online feed count,



The image shows a 'Create account' form for QueueVision. It features a dark blue background with a light blue header area. At the top, there is a user icon and the title 'Create account'. Below the title, a subtitle reads 'Register a new manager account for QueueVision'. The form consists of four input fields: 'Display Name' (placeholder: 'Your name'), 'Email' (placeholder: 'manager@queuevision.com'), 'Password' (placeholder: 'Create a strong password'), and 'Confirm Password' (placeholder: 'Re-enter your password'). Each password field has an eye icon for toggling visibility. A large blue 'Create account' button is positioned below the confirm password field. At the bottom, a link says 'Already have an account? Sign in'.

Create account

Register a new manager account for QueueVision

Display Name

Your name

Email

manager@queuevision.com

Password

Create a strong password

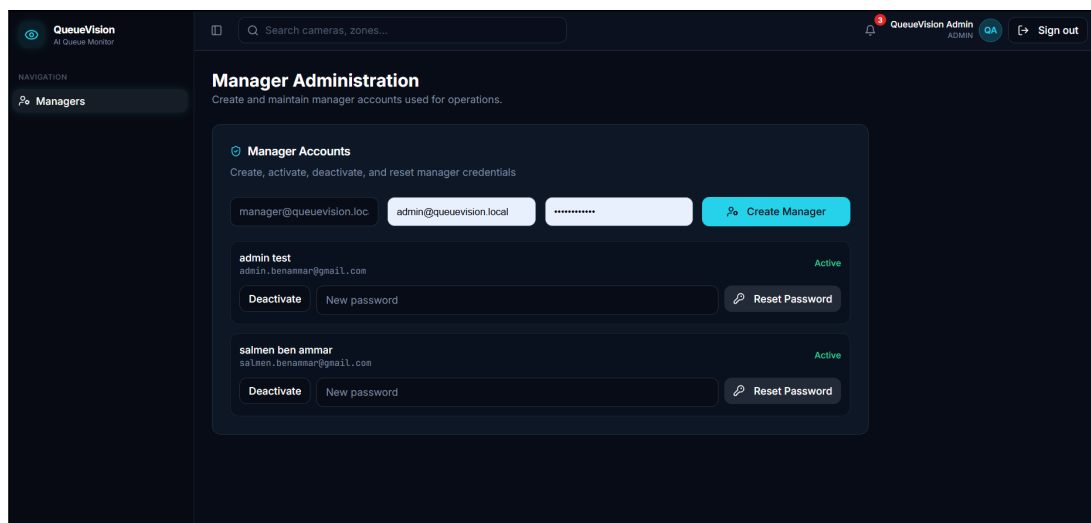
Confirm Password

Re-enter your password

Create account

Already have an account? [Sign in](#)

Figure 5.3: Manager account creation screen.



The image shows the 'Manager Administration' interface in the QueueVision dashboard. The interface has a dark blue theme. On the left, there is a sidebar with a 'QueueVision AI Queue Monitor' logo and a 'Managers' link. The main content area is titled 'Manager Administration' and includes a subtitle 'Create and maintain manager accounts used for operations.' Below this, there is a section for 'Manager Accounts' with a description 'Create, activate, deactivate, and reset manager credentials'. This section contains three rows of account information. The first row shows a 'Create Manager' button. The second and third rows show existing accounts with 'Deactivate' and 'Reset Password' buttons. The top right of the interface shows the user 'QueueVision Admin' and a 'Sign out' button.

QueueVision
AI Queue Monitor

Search cameras, zones...

QueueVision Admin QA Sign out

Managers

Manager Administration
Create and maintain manager accounts used for operations.

Manager Accounts
Create, activate, deactivate, and reset manager credentials

manager@queuevision.loc admin@queuevision.local Create Manager

admin test
admin.benamar@gmail.com Active
Deactivate New password Reset Password

salmen ben ammar
salmen.benamar@gmail.com Active
Deactivate New password Reset Password

Figure 5.4: Administrator manager-administration interface.

total people, average wait time, and alert count for visualization.

Live reliability relies on the websocket integration layer with multi-layered resilience:

- reconnection delays to prevent backend overload during outages,
- heartbeat monitoring for early connection loss detection,
- stale data guards to prevent outdated metric display,
- fallback polling when live events become unavailable,
- batched updates for interface responsiveness.

When websocket drops, the dashboard automatically switches to polling, maintaining manager visibility. In a monitoring context, operational continuity outweighs transport purity—the interface must remain informative despite network transients. This strategy preserves monitoring surface availability across degraded or interrupted connections.

5.4.6 Live Video Delivery and Transport Selection

The frontend intelligently selects video transport based on source type and availability: WebRTC for IP cameras when available (providing lower latency), and MJPEG for local MP4 files. Each running feed checks transport readiness and routes accordingly.

For IP camera feeds over WebRTC, the browser creates a receive connection, gathers network information, sends an offer to the backend, and applies the returned response to transition to live playback. Reconnection uses increasing delays to avoid backend flooding. When primary transport fails, the system stops retrying and switches to fallback. For IP feeds losing WebRTC connectivity, MJPEG fallback maintains visibility and operational continuity, ensuring monitoring does not depend on a single transport mechanism.

5.4.7 Dashboard UI Integration

The dashboard integrates live feed controls, setup dialogs, onboarding, KPI visualizations, and charts. Onboarding supports both uploaded videos and ONVIF sources. For cameras, the manager discovers devices, supplies per-camera credentials, resolves RTSP streams, validates the selection, captures a snapshot for zone definition, and stages the feed for launch. Figure 5.5 shows the operational monitoring surface.

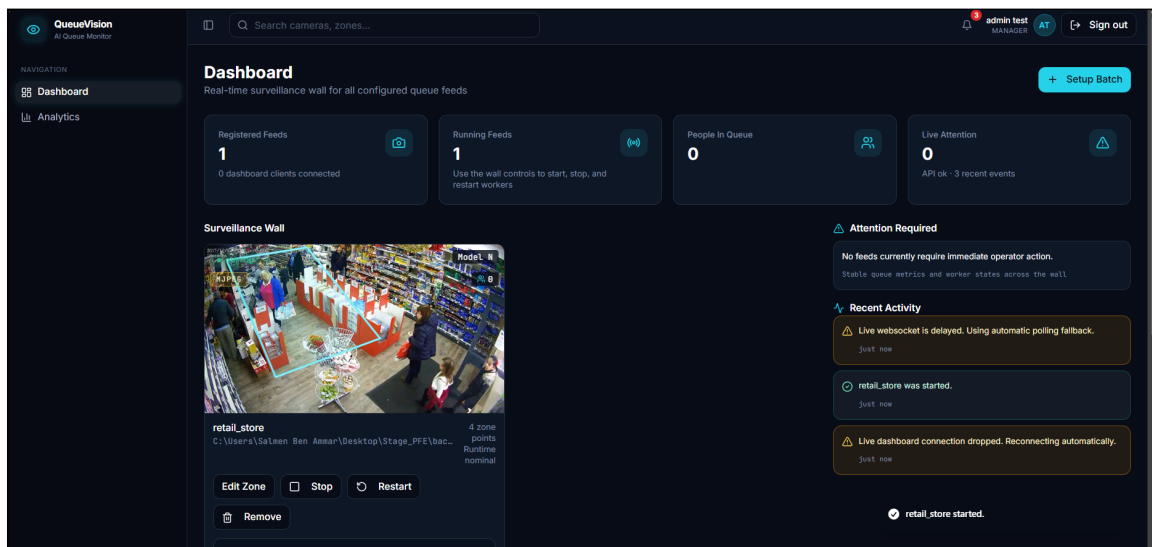


Figure 5.5: Manager dashboard: live surface, feed summaries, and recent activity.

5.4.8 Manager Onboarding Workflow

The user-action flow for the manager onboarding process captures the complete journey from setup wizard launch through feed confirmation. Figure 5.6 illustrates the workflow phases: source selection (ONVIF camera or upload), source configuration with connection testing and retry logic, zone definition and model selection, and finally review and launch confirmation.

The workflow execution involves several screens. First, the feed setup screen prepares an uploaded video or ONVIF source:

Then the zone is defined through the tracing interface:

Model selection precedes the final confirmation. The manager chooses the YOLO

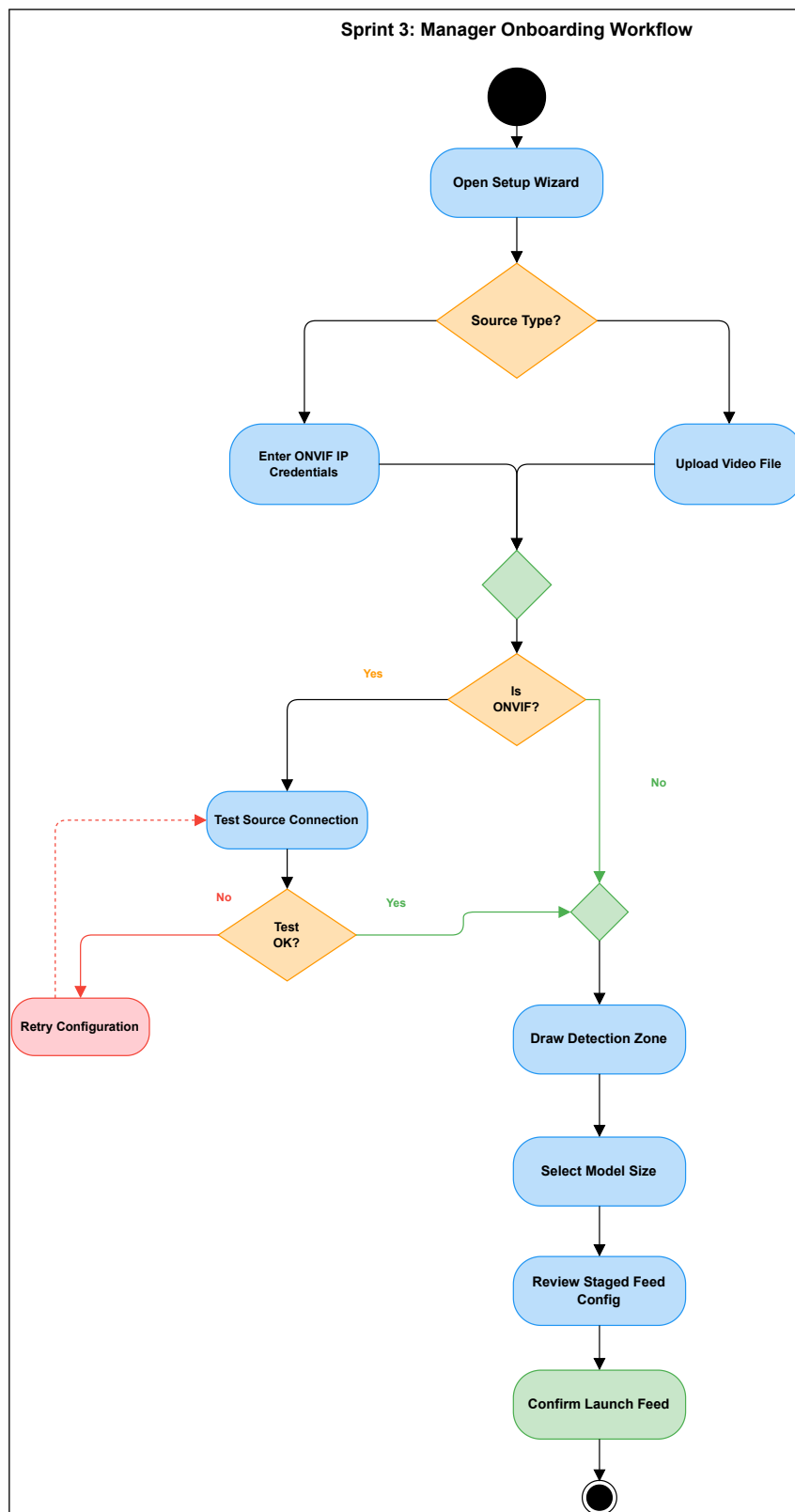


Figure 5.6: Manager onboarding activity diagram showing user actions and UI state transitions from setup wizard initialization through feed launch confirmation. Decision nodes show ONVIF versus upload path selection and the test validation retry loop.

Stage Sources ×

Enter the feed information for one source, optionally attach metadata, then continue through zone tracing, model selection, and review.

One source is prepared at a time. Finish this source, then continue to zone tracing and model selection.

Feed name

Checkout 1

Source type

Local MP4 ONVIF Discovery

Upload a local video
Supported formats: MP4, MOV, AVI, and MKV. The backend stores the file and uses its path automatically.

Choose Video File No video selected yet

Local videos are configured one at a time so each feed keeps a single zone and model.

Step 1 uploads the source file into the backend workflow. Step 2 lets you draw the queue polygon on the video frame. Step 3 selects the model size.

Pick one local video to see it listed here.

Cancel Continue to Zone

Figure 5.7: Feed setup screen used to prepare an uploaded video or ONVIF camera source before zone tracing and model selection.

size here:

Finally, the staged feed or batch is confirmed before launch:

The practical result is that a manager can move from onboarding to monitoring without leaving the dashboard context, keeping setup, validation, launch, and supervision inside the same manager-facing interface.

5.4.9 Analytics and Historical Reporting

The analytics view complements live supervision with historical reporting. It aggregates archived alerts and queue metrics so managers can spot trends and bottlenecks without leaving the dashboard.

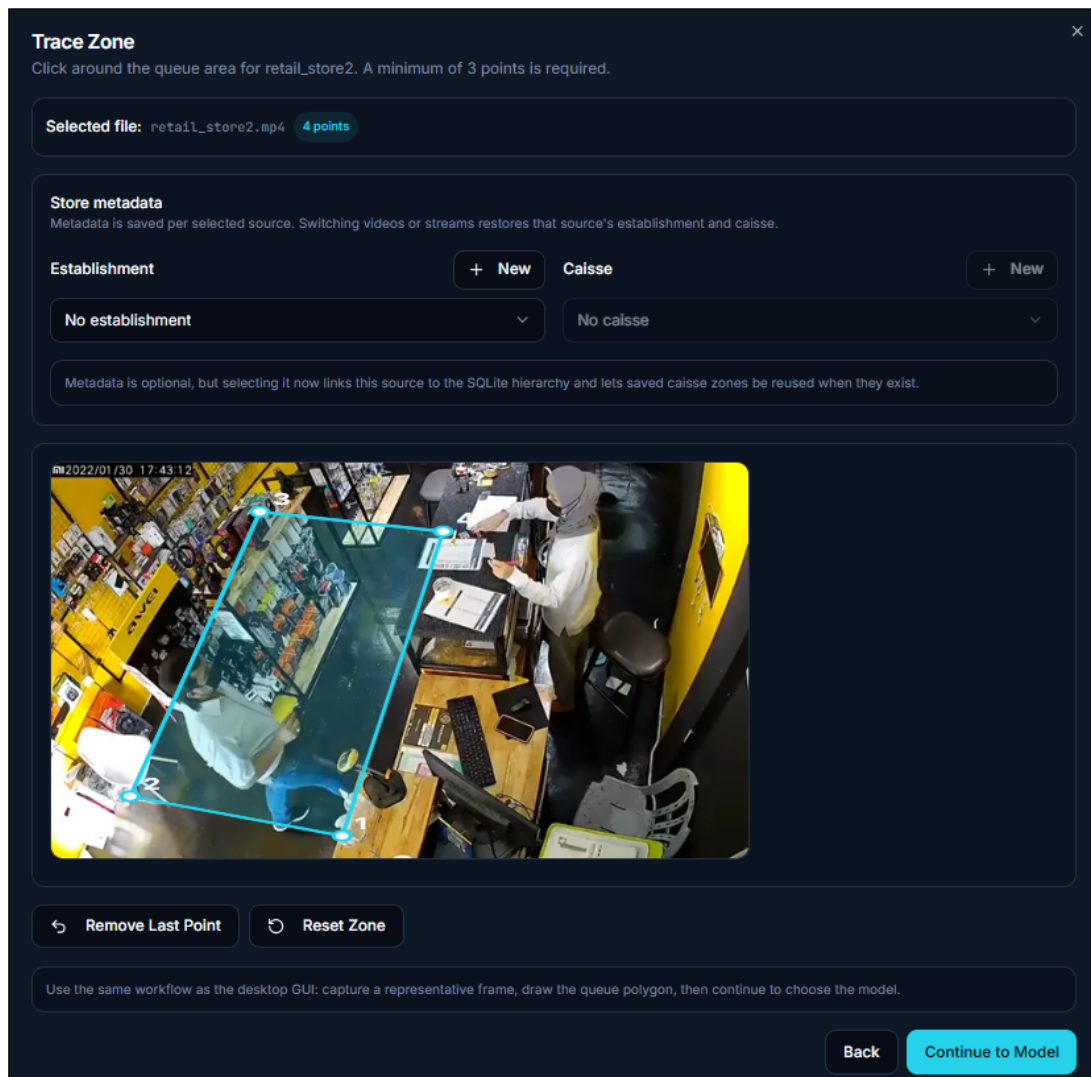


Figure 5.8: Zone tracing screen used to define the queue polygon for the selected feed.

Four KPI cards summarize: average wait time, peak queue length, stability score, total alerts. A period selector (hourly, daily, weekly) controls granularity; date filters isolate specific windows. The dashboard visualizes wait and queue trends, ranks top and bottom zones, and breaks down alerts by severity. The view also supports CSV export for external reporting. During early pilots when the archive is empty, the interface uses mock data derived from active feeds so the analytics are still meaningful.

Figure 5.11 shows the analytics dashboard.

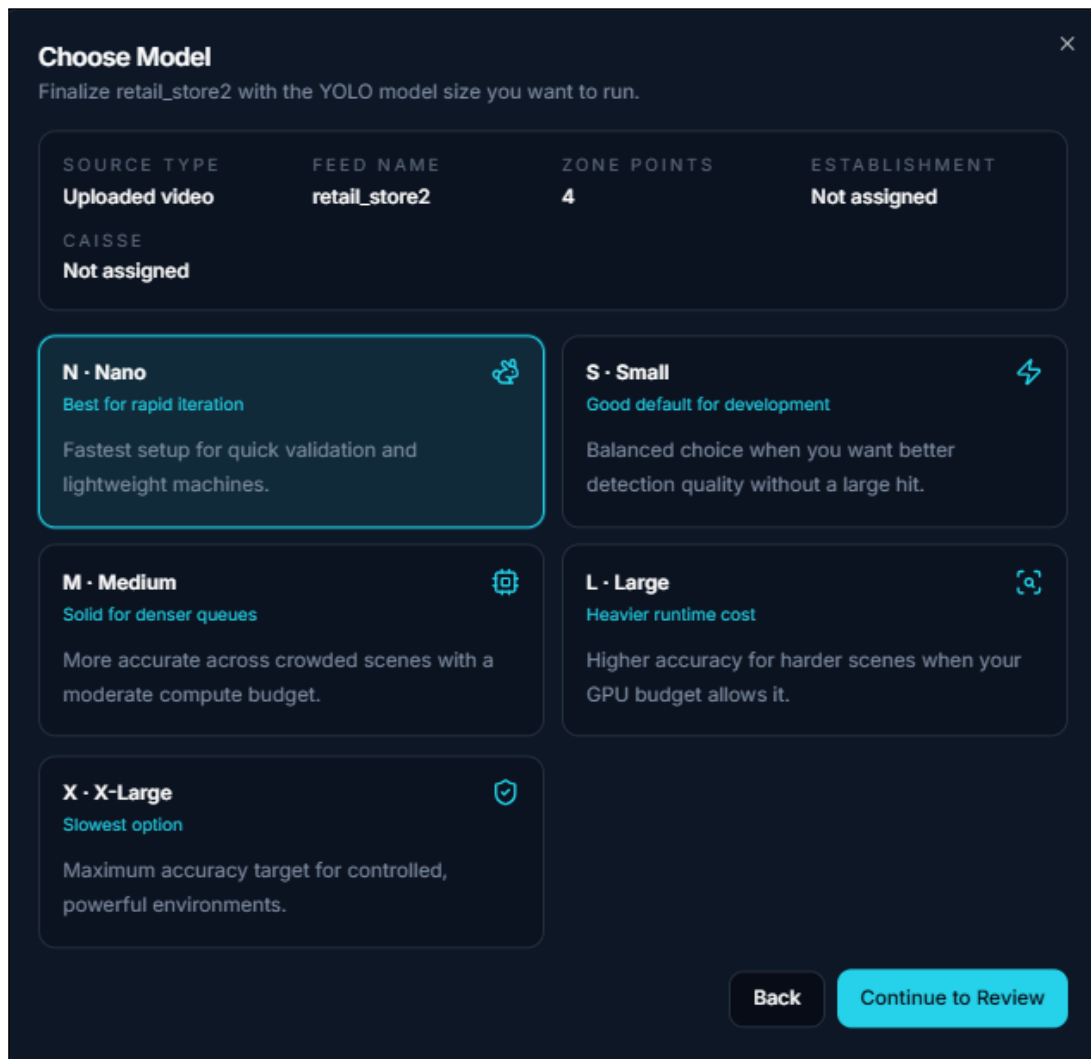


Figure 5.9: Model selection screen used to choose the YOLO size before launching the feed.

5.4.10 Integration Testing

We tested the dashboard through scenarios: setup flows, live event handling, transport fallback, feed routing, and error handling.

The tests do more than catch regressions—they document what the sprint actually delivers. A rendered screen is not enough; the dashboard had to work correctly when websockets reconnected, routes changed, and fallbacks activated.

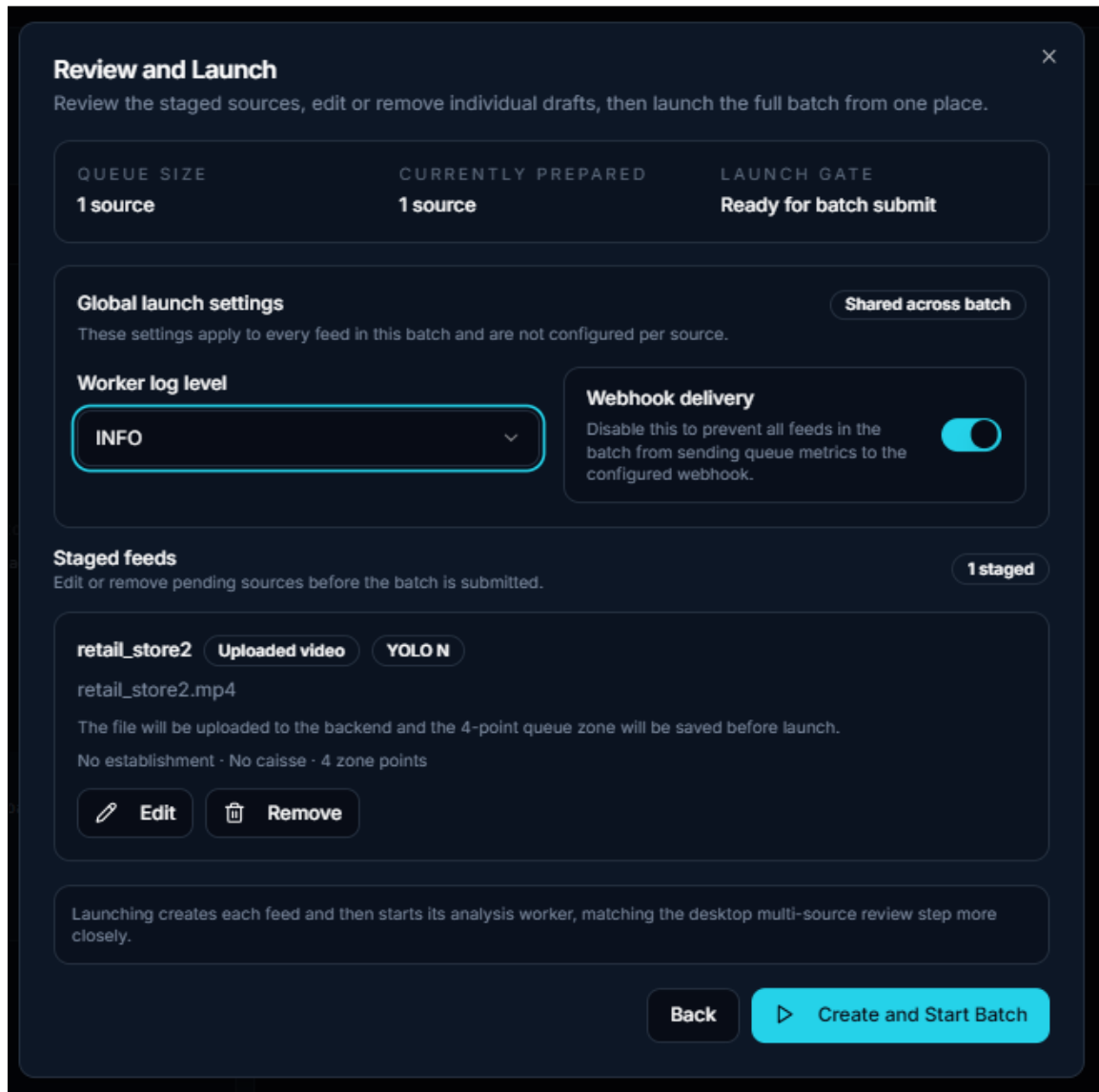


Figure 5.10: Review and launch panel used to confirm staged feeds before batch creation or launch.

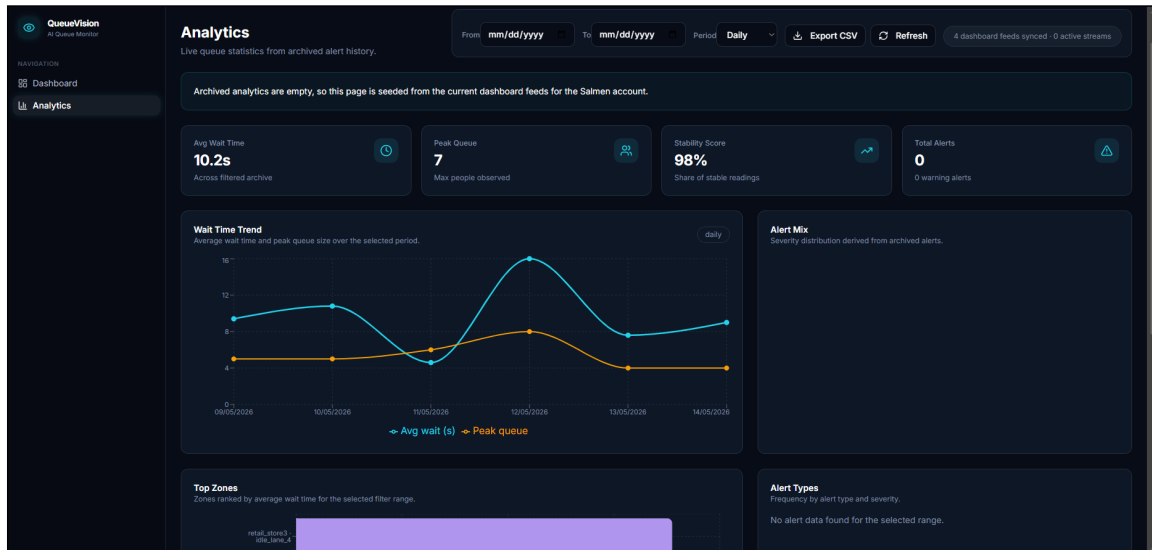


Figure 5.11: Analytics dashboard: KPI summaries, trends, zone rankings, and export.

5.5 Conclusion

Sprint 3 completed the frontend implementation and integrated it with the backend to create a fully operational dashboard. The design-first approach proved effective: by settling the visual language, page structure, and component boundaries before implementation, the integration phase could focus entirely on behavioral correctness and transport resilience.

The key architectural achievement was the state orchestration layer that unified session state, feed control actions, live metric updates, and multi-layered fallback logic into a coherent operator experience. The dashboard remained usable across network transients by implementing WebSocket reconnection with exponential backoff, stale-data guards, automatic fallback to polling, and transport negotiation—ensuring that monitoring continuity does not depend on any single mechanism. This resilience strategy proved essential for operational deployments where transient failures are inevitable.

By the end of the sprint, the frontend was complete and production-ready for alert automation and comprehensive system validation in Sprint 4.

Sprint 4: Automation, Evaluation, and Optimization

Table of Content

6.1	Introduction	62
6.2	Sprint Objectives and Backlog	62
6.2.1	Technical Tasks	63
6.3	Design	64
6.3.1	Alert Automation and n8n Workflow	64
6.3.2	System Verification and Evaluation Strategy	65
6.4	Implementation	67
6.4.1	Alert Payload and Delivery Architecture	67
6.4.2	Telegram Integration and Notification Delivery	67
6.4.3	Asynchronous Alert Dispatch and Deduplication	69
6.4.4	n8n Workflow Structure and Validation	69
6.4.5	Comprehensive Integration Verification	69
6.4.6	Runtime Optimization and Performance Tuning	70
6.4.7	Fault-Tolerance and Recovery Mechanisms	70
6.4.8	System Integration and Operational Readiness	70
6.5	Conclusion	71

6.1 Introduction

Sprint 4 completed the system by integrating alert automation and conducting comprehensive end-to-end validation. The sprint connected queue metrics from the backend to operational staff through Telegram notifications via n8n, then validated that the complete system—perception engine, API, dashboard, and alert automation—functioned correctly under integrated conditions.

This chapter covers both the alert automation implementation and the verification strategy that ensured system reliability and operational viability.

6.2 Sprint Objectives and Backlog

Sprint 4 pursued two parallel tracks: implementing alert automation and conducting system-wide verification. Alert automation required designing a validated webhook payload structure, implementing delivery reliability through retry logic and monitoring, configuring n8n to handle authorization and cooldown logic, and routing alerts to Telegram. Verification required running backend and frontend tests in integrated scenarios, validating API contracts and streaming stability, improving system responsiveness through runtime tuning, and strengthening fault tolerance for transient failures. The sprint backlog organized these objectives into manager-facing user stories and technical implementation tasks. The two main manager needs were alert automation and reliable notification delivery. Table 6.1 lists the user stories that drove this work.

User Story ID	User Story	Task ID	Task
US4.1	As a manager, I want alert payloads dispatched to n8n so that operational staff can be notified automatically.	T4.1	Define the webhook payload schema and validate it before dispatch.
US4.3	As a manager, I want alerts routed through Telegram so that the on-site operator receives notifications on a familiar channel.	T4.3	Create and configure the Telegram bot, store its credential in n8n, and configure the workflow.

Table 6.1: Sprint 4 user stories

6.2.1 Technical Tasks

Beyond the user stories, we had to ensure reliability, testing, and performance improvements. The technical work breakdown is shown in Table 6.2.

Task ID	Task	Subtask ID	Subtask
T4.2	Add retry logic, timeout handling, and logging for webhook delivery to ensure notifications are not silently lost.		Implement delivery resilience with exponential backoff and failure tracking.
T4.4	Prepare PowerShell scripts for single-alert, multi-feed, and execution polling scenarios to validate n8n workflow behavior.		Create repeatable test automation for workflow routing and alerting paths.
T4.5	Run backend and frontend tests together and confirm lifecycle, streaming, and API contracts under realistic conditions.		Execute comprehensive integration verification to catch issues before delivery.
T4.6	Apply runtime tuning: configurable image size, frame stride, async dispatch, and playback pacing to improve processing responsiveness.		Implement performance optimizations for the main processing loop.
T4.7	Improve feed worker supervision, frame handoff continuity, and websocket reconnection to strengthen fault tolerance.		Enhance system recovery from transient failures and network disruptions.

Table 6.2: Sprint 4 technical tasks

6.3 Design

6.3.1 Alert Automation and n8n Workflow

The alert workflow implements a multi-stage pipeline designed to prevent alert fatigue while ensuring critical notifications reach operational staff. Upon receipt of a queue alert from the backend, the workflow validates the webhook secret to confirm message authenticity, applies cooldown logic to suppress repeated alerts from the same threshold

violation, archives the alert to the persistence layer if retention is required, and routes the validated alert to the Telegram notification channel. This design separates authentication, deduplication, and delivery concerns, allowing each stage to be independently tested and debugged. Figure 6.1 illustrates the complete workflow logic.

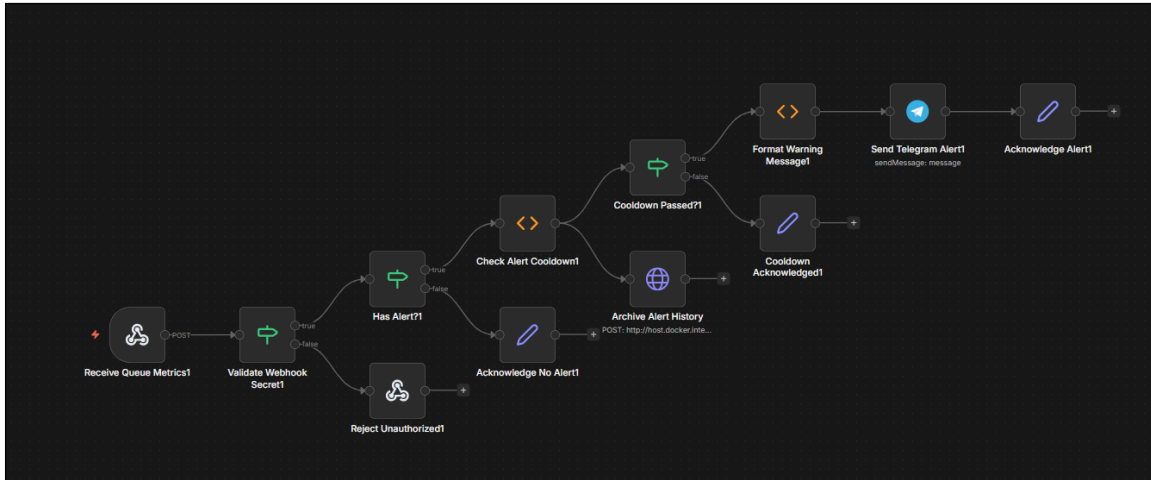


Figure 6.1: Sprint 4 n8n workflow for routing queue alerts through cooldown checks, archival, and Telegram delivery.

6.3.2 System Verification and Evaluation Strategy

The verification approach employed a layered testing strategy that addressed backend analytics and API correctness, frontend dashboard responsiveness and fallback behavior, and workflow routing logic across n8n branches. Critical measurements included perception pipeline latency (frames processed per second), streaming stability during feed restart, alert delivery time from threshold detection to Telegram, and dashboard responsiveness when WebSocket connections were degraded or lost.

The evaluation methodology combined unit-level testing of individual components with integration testing that verified inter-component contracts under realistic conditions. Test scenarios encompassed normal operation, transient failure recovery, feed lifecycle management, and alerting under varying queue states. This comprehensive approach ensured that optimization and fault-tolerance improvements did not introduce regressions in other system layers.

Figure 6.2 illustrates the iterative evaluation and tuning cycle that characterized the sprint.

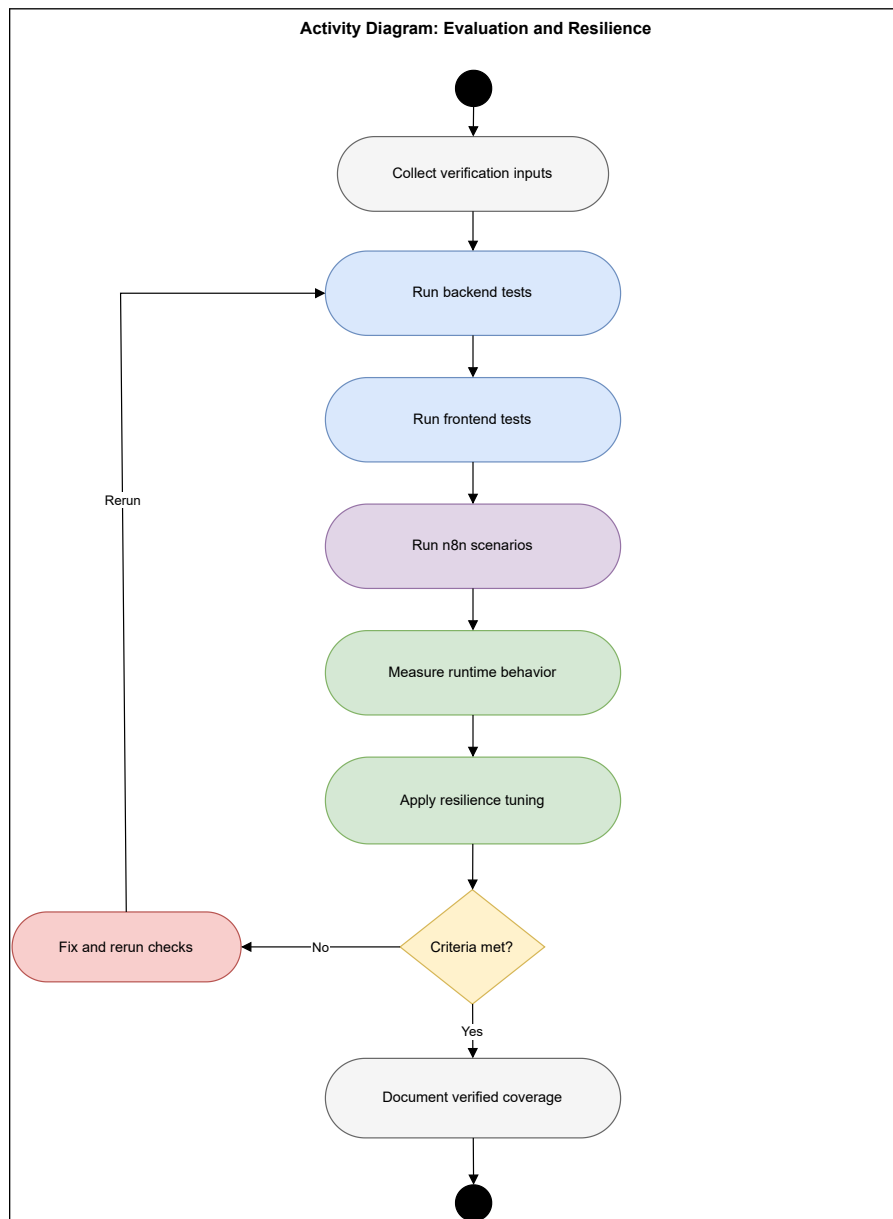


Figure 6.2: Sprint 4 evaluation and resilience activity diagram for tests, tuning, and recovery verification.

6.4 Implementation

6.4.1 Alert Payload and Delivery Architecture

The alert delivery system defines a structured ‘QueuePayload’ class that encapsulates all information necessary for n8n processing and Telegram notification: timestamp and frame metadata, queue metrics (arrival rate λ , service rate μ , expected wait time), alert flags, and severity classification. Payload validation occurs before dispatch to ensure the backend never transmits malformed or incomplete messages to external systems.

Delivery reliability is ensured through several mechanisms: retry logic with exponential backoff for transient failures, timeout-aware scheduling to prevent webhook hangups, secure shared-secret validation to confirm backend authenticity, and comprehensive logging of all dispatch attempts. All webhook payloads, whether successfully delivered or failed, are persisted to the database for audit trails and failure analysis. This architecture ensures that alert loss is detectable and that operational staff can review the complete history of threshold violations and notification attempts.

6.4.2 Telegram Integration and Notification Delivery

A Telegram bot was configured as the notification channel, with authentication credentials stored securely in n8n’s credential management system rather than hardcoded in workflow exports. This separation ensures portability and prevents secret exposure during workflow sharing or version control. Alerts passing validation and cooldown checks are formatted for human readability and dispatched to a designated operational group chat. An example of delivered alert formatting is shown in Figure 6.3.

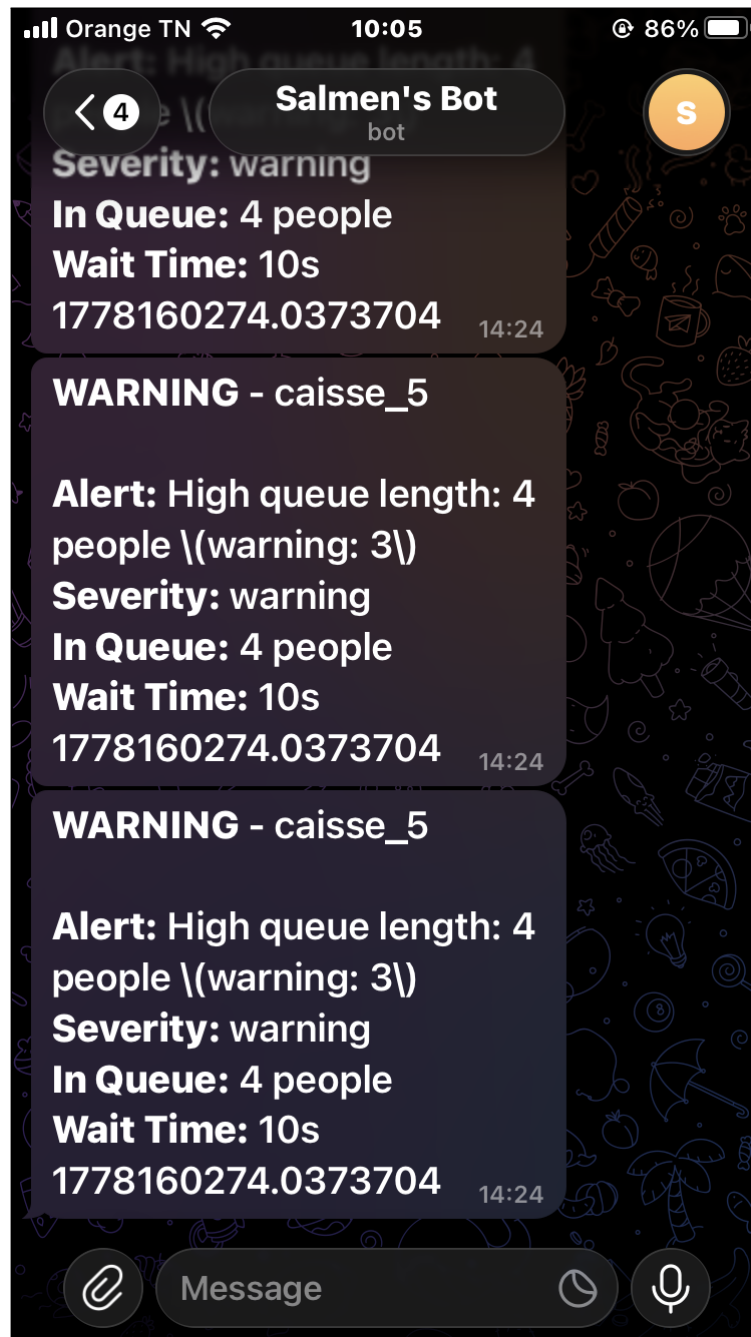


Figure 6.3: Example Telegram alert delivered.

6.4.3 Asynchronous Alert Dispatch and Deduplication

Alert dispatch is decoupled from the main perception pipeline through asynchronous processing: an 'AsyncWebhookDispatcher' thread handles webhook transmission without blocking the detection and analytics loop. This architecture prevents alert delivery latency from impacting perception frame rate. Deduplication logic identifies and suppresses duplicate alerts when the same threshold violation persists across consecutive metric windows, reducing alert noise for repeated conditions while ensuring that new threshold violations still trigger notifications.

6.4.4 n8n Workflow Structure and Validation

The n8n workflow implementation separates three distinct branches corresponding to different alert states: no alert condition (immediate return), suppressed alert due to cooldown (logged but not delivered), and delivery-ready alert (sent to Telegram). This modular structure enables independent testing and debugging of each routing path. Automated validation using PowerShell test scripts verified workflow behavior across single-alert scenarios, multi-feed deployments, and sustained alert streams, ensuring that cooldown logic, authorization checks, and Telegram delivery functioned correctly across diverse operational conditions.

6.4.5 Comprehensive Integration Verification

Verification testing covered the complete system stack: API contract validation across all endpoints, streaming stability during feed lifecycle transitions, webhook delivery under normal and degraded conditions, dashboard responsiveness during live updates, and end-to-end alert routing from threshold detection to Telegram notification. Error scenarios included authentication failures, WebSocket disconnections, WebRTC transport unavailability, and worker process crashes. This comprehensive testing approach

ensured that the system maintained operational correctness across failure modes that would realistically occur in production deployments.

6.4.6 Runtime Optimization and Performance Tuning

Several optimization strategies were implemented to improve system responsiveness and throughput. Image size and frame stride parameters were made configurable, allowing operators to trade detection accuracy for processing speed based on deployment constraints. GPU/CPU device selection was exposed as a configuration parameter to accommodate heterogeneous hardware. Webhook dispatch was moved to asynchronous execution to prevent network latency from blocking the main perception loop. Video playback timing was adjusted to simulate real-time frame delivery rather than processing files at maximum throughput, preventing downstream queue buffer overflow.

6.4.7 Fault-Tolerance and Recovery Mechanisms

System resilience against transient failures was enhanced through multiple recovery layers. The ‘FeedRegistry’ component implements automatic feed worker restart when process failures are detected, preventing orphaned feeds from blocking new sources. Frame handoff between camera capture and worker threads was hardened to prevent frame loss during queue transitions. Frontend WebSocket reconnection was implemented with automatic fallback to periodic polling, ensuring that dashboard monitoring capability is maintained even when real-time event delivery is unavailable.

6.4.8 System Integration and Operational Readiness

By the conclusion of Sprint 4, all major system components—perception pipeline, queue analytics, backend API, web dashboard, and n8n alert automation—integrated successfully into a cohesive platform. Comprehensive testing verified that the system

remained stable and responsive under realistic operational scenarios. The primary deliverables remaining for production deployment were performance testing under sustained load and infrastructure hardening, rather than implementation of missing core features.

6.5 Conclusion

Sprint 4 delivered end-to-end system integration and operational validation. The alert automation pipeline was completed with secure webhook dispatch, cooldown-based deduplication, and Telegram notification delivery through n8n. Comprehensive verification testing across all system layers confirmed that perception, analytics, API, dashboard, and alert automation functioned correctly in integrated scenarios.

The sprint demonstrated that complex multi-layer systems require thorough end-to-end testing to expose integration issues that component-level testing cannot reveal. Alert delivery, streaming stability, and dashboard responsiveness under network transients emerged as critical success factors during verification. By addressing these concerns across all layers of the architecture, the sprint transformed the system from a collection of functional components into an operationally viable platform.

The system is now positioned for production deployment with performance validation and infrastructure hardening as the remaining steps before full operational readiness.

General Conclusion

Table of Content

7.1	Summary	73
7.2	Objective Review	73
7.3	Future Work	74

7.1 Summary

This end-of-studies project produced a complete queue wait-time estimation platform that brings together computer vision, queue analytics, web supervision, and automation workflows.

In practical terms, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- adaptive wait-time outputs that respond to queue conditions,
- a secure authentication layer for manager sign-up, sign-in, session refresh, and administrator account governance,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live web-socket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

The resulting architecture shows that a full monitoring-to-action loop can be built from modular components without losing maintainability or testability.

Across the four sprint chapters, the work progresses from perception and queue analytics to backend services, then to the web dashboard, and finally to automation and validation.

7.2 Objective Review

The initial objectives were achieved at the project scope level:

1. **Real-time detection and tracking:** achieved through dedicated detector and

tracker modules.

2. **Queue-zone counting:** achieved with polygon-zone management and in-zone identity filtering.
3. **Queue metric estimation:** achieved for arrival rate, service rate, and wait time.
4. **Adaptive wait-time estimation:** achieved through automatic formula switching based on queue load.
5. **Web monitoring:** achieved through a multi-feed dashboard with live updates.
6. **Automation integration:** achieved through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but the depth of long-horizon evaluation under broader operational scenarios.

7.3 Future Work

Possible extensions include:

- multi-zone and multi-server queue modeling for parallel cashier configurations,
- richer alert strategies with cooldown tuning, escalation policies, and acknowledgment loops,
- edge deployment optimization through ONNX/TensorRT profiling and resource-aware scheduling,
- long-duration field evaluation with domain-specific KPI baselines,
- stronger security controls for production webhook exposure and secrets management.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation – yolo real-time object detection*, <https://docs.ultralytics.com/>, Accessed: 2026-03-01, 2026.
- [3] Waitwhile, *Waitwhile – virtual queue and appointment management*, <https://www.waitwhile.com/>, Accessed: 2026-02-24, 2026.
- [4] Qminder, *Qminder – queue management system*, <https://www.qminder.com/>, Accessed: 2026-02-24, 2026.
- [5] RetailNext, *Retailnext – in-store analytics and traffic intelligence*, <https://retailnext.net/>, Accessed: 2026-02-25, 2026.
- [6] Xovis, *Xovis – 3d sensor-based people flow and queue measurement*, <https://www.xovis.com/>, Accessed: 2026-02-25, 2026.
- [7] Roboflow, *Roboflow – computer vision platform*, <https://roboflow.com/>, Accessed: 2026-02-26, 2026.
- [8] Microsoft, *Visual studio code documentation*, <https://code.visualstudio.com/docs>, Accessed: 2026-02-21, 2026.
- [9] Git, *Git documentation*, <https://git-scm.com/doc>, Accessed: 2026-02-20, 2026.
- [10] GitHub, *Github documentation*, <https://docs.github.com/>, Accessed: 2026-02-20, 2026.
- [11] diagrams.net, *Diagrams.net (draw.io) documentation*, <https://www.diagrams.net/>, Accessed: 2026-02-23, 2026.

REFERENCES

- [12] Docker, *Docker documentation – containers and compose*, <https://docs.docker.com/>, Accessed: 2026-02-19, 2026.
- [13] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [14] OpenCV, *OpenCV documentation*, <https://docs.opencv.org/>, Accessed: 2026-02-27, 2026.
- [15] NumPy, *Numpy documentation*, <https://numpy.org/doc/>, Accessed: 2026-02-25, 2026.
- [16] pandas, *Pandas documentation*, <https://pandas.pydata.org/docs/>, Accessed: 2026-04-01, 2026.
- [17] SciPy, *Scipy documentation*, <https://docs.scipy.org/doc/scipy/>, Accessed: 2026-04-03, 2026.
- [18] S. Ramírez, *Fastapi documentation*, <https://fastapi.tiangolo.com/>, Accessed: 2026-03-17, 2026.
- [19] Pydantic, *Pydantic documentation*, <https://docs.pydantic.dev/>, Accessed: 2026-03-18, 2026.
- [20] W3C Web Real-Time Communications Working Group, *WebRTC 1.0: Real-time communication between browsers*, <https://www.w3.org/TR/webrtc/>, Accessed: 2026-03-27, 2025.
- [21] MediaMTX, *Mediamtx – media server documentation*, <https://github.com/bluenvion/mediamtx>, Accessed: 2026-03-24, 2026.
- [22] React, *React documentation*, <https://react.dev/>, Accessed: 2026-04-14, 2026.
- [23] Microsoft, *Typescript documentation*, <https://www.typescriptlang.org/docs/>, Accessed: 2026-04-15, 2026.
- [24] Vite, *Vite documentation*, <https://vite.dev/guide/>, Accessed: 2026-04-16, 2026.

REFERENCES

- [25] T. CSS, *Tailwind css documentation*, <https://tailwindcss.com/docs/>, Accessed: 2026-04-18, 2026.
- [26] n8n, *N8n documentation – workflow automation*, <https://docs.n8n.io/>, Accessed: 2026-05-13, 2026.
- [27] Telegram, *Telegram bot api*, <https://core.telegram.org/bots/api>, Accessed: 2026-05-15, 2026.
- [28] SQLite, *Sqlite documentation*, <https://www.sqlite.org/docs.html>, Accessed: 2026-03-20, 2026.
- [29] Internet Engineering Task Force, *Rfc 2326: Real time streaming protocol (rtsp)*, <https://www.rfc-editor.org/rfc/rfc2326>, Accessed: 2026-03-08, 1998.
- [30] ONVIF, *Onvif specifications*, <https://www.onvif.org/profiles/specifications/>, Accessed: 2026-03-06, 2026.